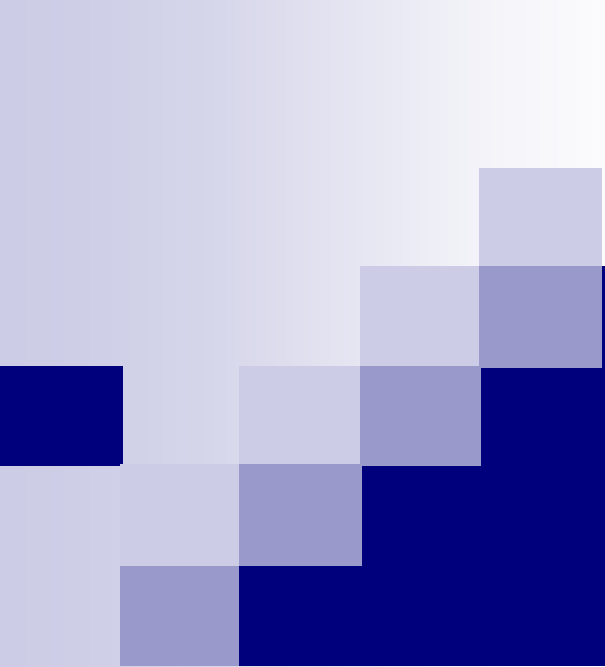
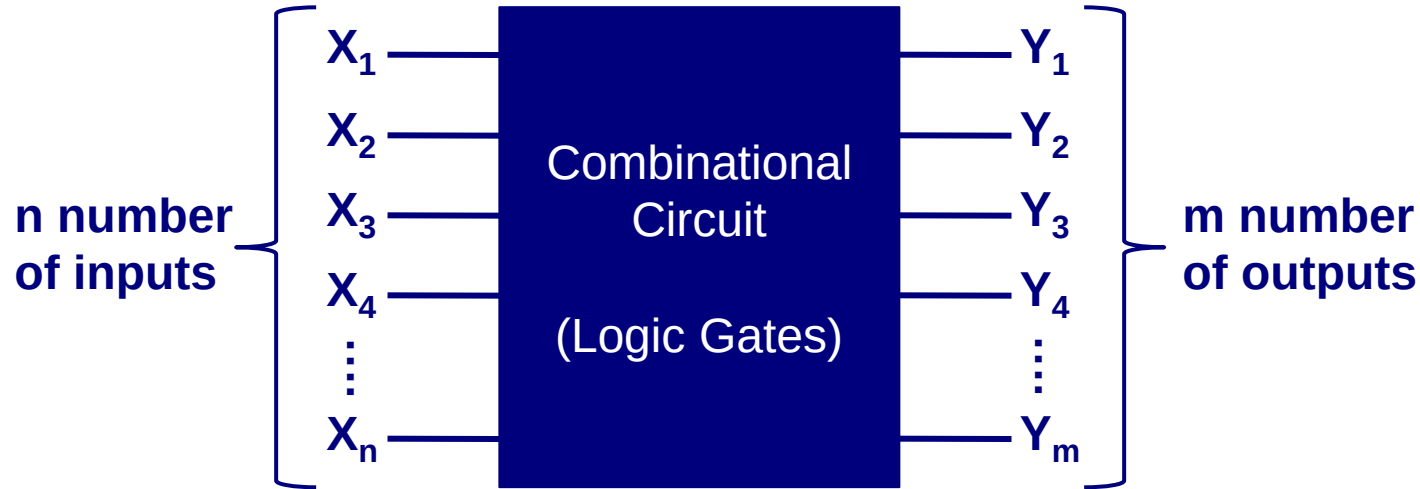




Combinational Circuits

Dr. Nilesh Patidar and Mr. Shiraz Husain

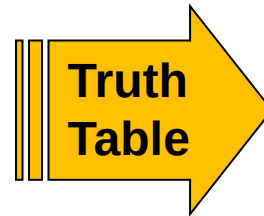
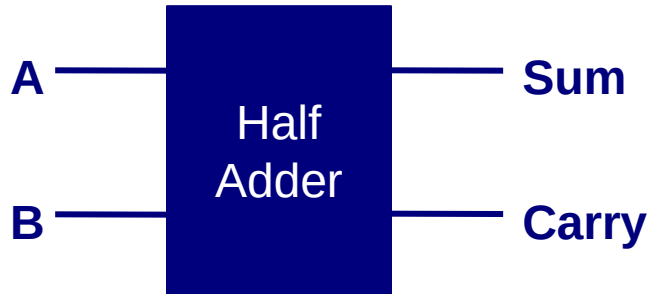
Combinational Circuits



Does not contain any memory element/feedback path

Half adder

- Half adder is used for addition of 2 bits.



A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half adder

K-map for Sum

$A \backslash B$	B' (0)	B (1)
A' (0)	0	1
A (1)	1	0

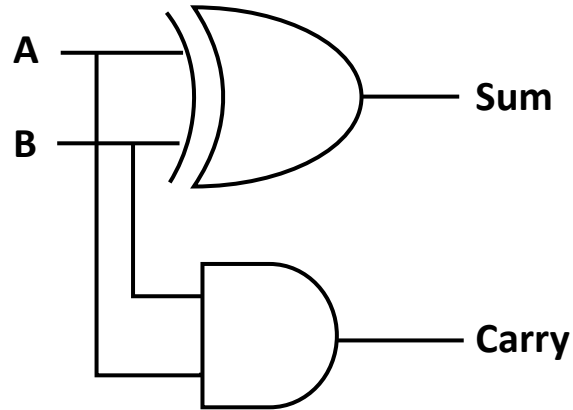
$$\begin{aligned}\text{Sum} &= AB' + A'B \\ &= A \oplus B\end{aligned}$$

K-map for Carry

$A \backslash B$	B' (0)	B (1)
A' (0)	0	0
A (1)	0	1

$$\text{Carry} = AB$$

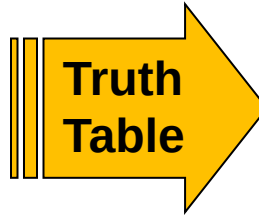
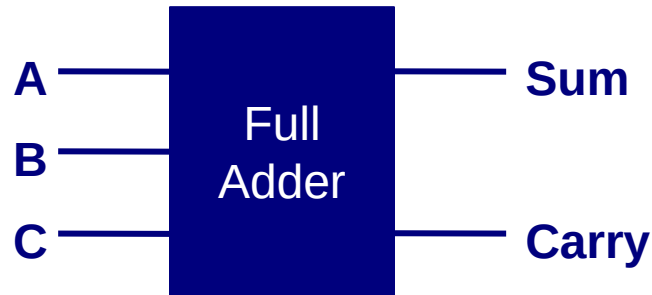
Half adder



Logic Circuit of Half adder

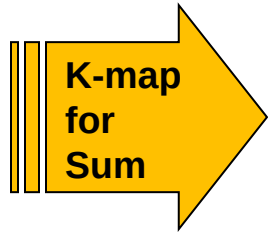
Full adder

- Full adder is used for addition of 3 bits.



A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full adder



$A \backslash BC$	$B'C' \text{ (00)}$	$B'C \text{ (01)}$	$BC \text{ (11)}$	$BC' \text{ (10)}$
$A' \text{ (0)}$	0 0	1 1	0 3	1 2
$A \text{ (1)}$	1 4	0 5	1 7	0 6

$$\begin{aligned}\text{Sum} &= AB'C' + ABC + A'B'C + A'BC' \\ &= A(B'C' + BC) + A'(B'C + BC') \\ &= A(B \oplus C)' + A'(B \oplus C) \\ &= A \oplus B \oplus C\end{aligned}$$

Full adder



$A \backslash BC$	$B'C' \text{ (00)}$	$B'C \text{ (01)}$	$BC \text{ (11)}$	$BC' \text{ (10)}$
$A' \text{ (0)}$	0 0	0 1	1 3	0 2
$A \text{ (1)}$	0 4	1 5	1 7	1 6

$\text{Carry} = AC + BC + AB$

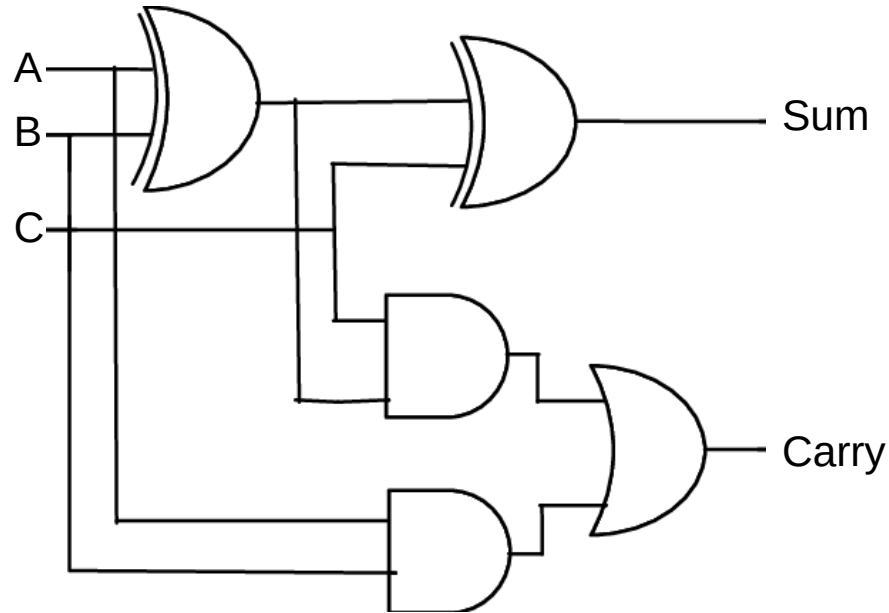
Full adder



$A \backslash BC$	$B'C' \text{ (00)}$	$B'C \text{ (01)}$	$BC \text{ (11)}$	$BC' \text{ (10)}$
$A' \text{ (0)}$	0 0	0 1	1 3	0 2
$A \text{ (1)}$	0 4	1 5	1 7	1 6

$$\begin{aligned}
 \text{Carry} &= AB'C + A'BC + AB \\
 &= (AB' + A'B)C + AB \\
 &= (A \oplus B)C + AB
 \end{aligned}$$

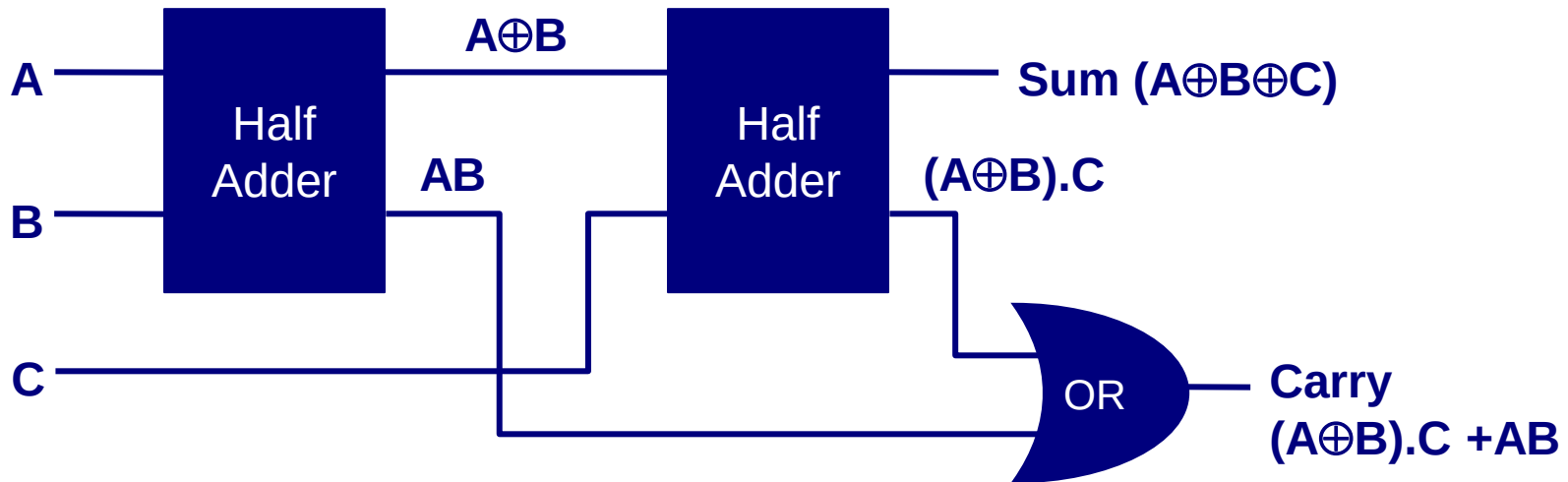
Full adder



logic circuit of full adder

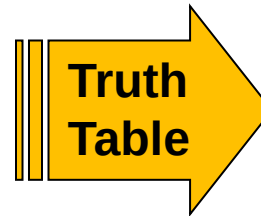
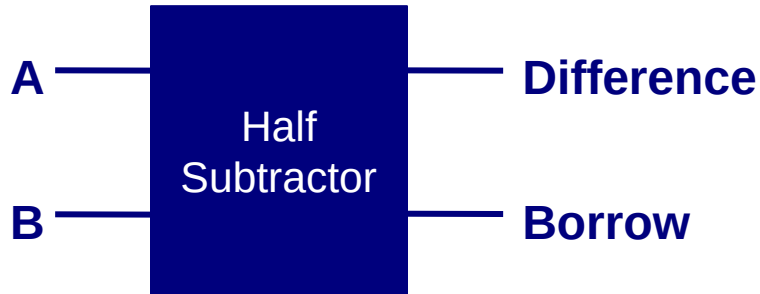
Design of Full adder using Half adders

- Full adder can be designed by using 2 half adders and 1 OR gate.



Half Subtractor

- Half Subtractor is used for subtraction of 2 bits.



A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Half subtractor

K-map for Difference

$A \backslash B$	B' (0)	B (1)
A' (0)	0	1
A (1)	1	0

$$\text{Difference} = AB' + A'B$$

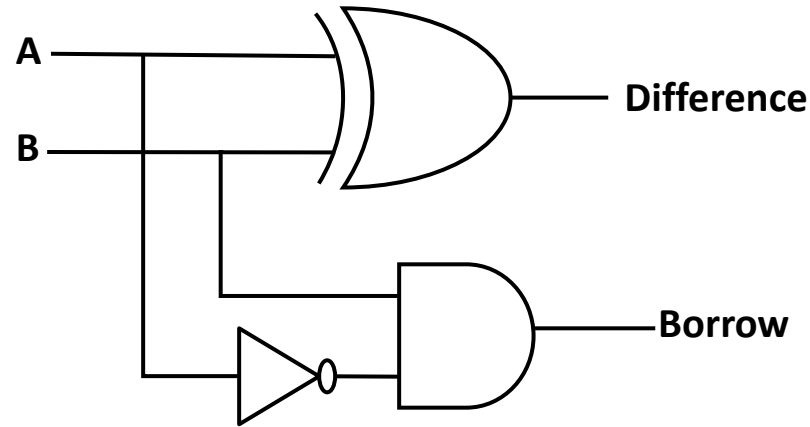
$$= A \oplus B$$

K-map for Borrow

$A \backslash B$	B' (0)	B (1)
A' (0)	0	1
A (1)	0	0

$$\text{Borrow} = A'B$$

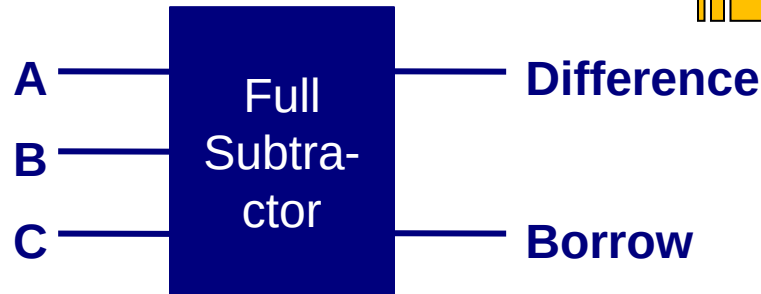
Half subtractor



Logic Circuit of Half subtractor

Full Subtractor

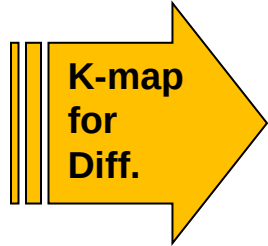
- Full Subtractor is used for subtraction of 3 bits.



Truth Table

A	B	C	Difference	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

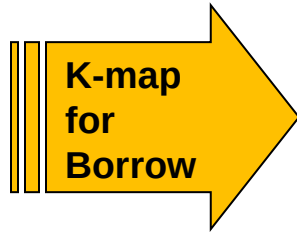
Full subtractor



$A \backslash BC$	$B'C' \text{ (00)}$	$B'C \text{ (01)}$	$BC \text{ (11)}$	$BC' \text{ (10)}$
$A' \text{ (0)}$	0 0	1 1	0 3	1 2
$A \text{ (1)}$	1 4	0 5	1 7	0 6

$$\begin{aligned}
 \text{Difference} &= AB'C' + ABC + A'B'C + A'BC' \\
 &= A(B'C' + BC) + A'(B'C + BC') \\
 &= A(B \oplus C)' + A'(B \oplus C) \\
 &= A \oplus B \oplus C
 \end{aligned}$$

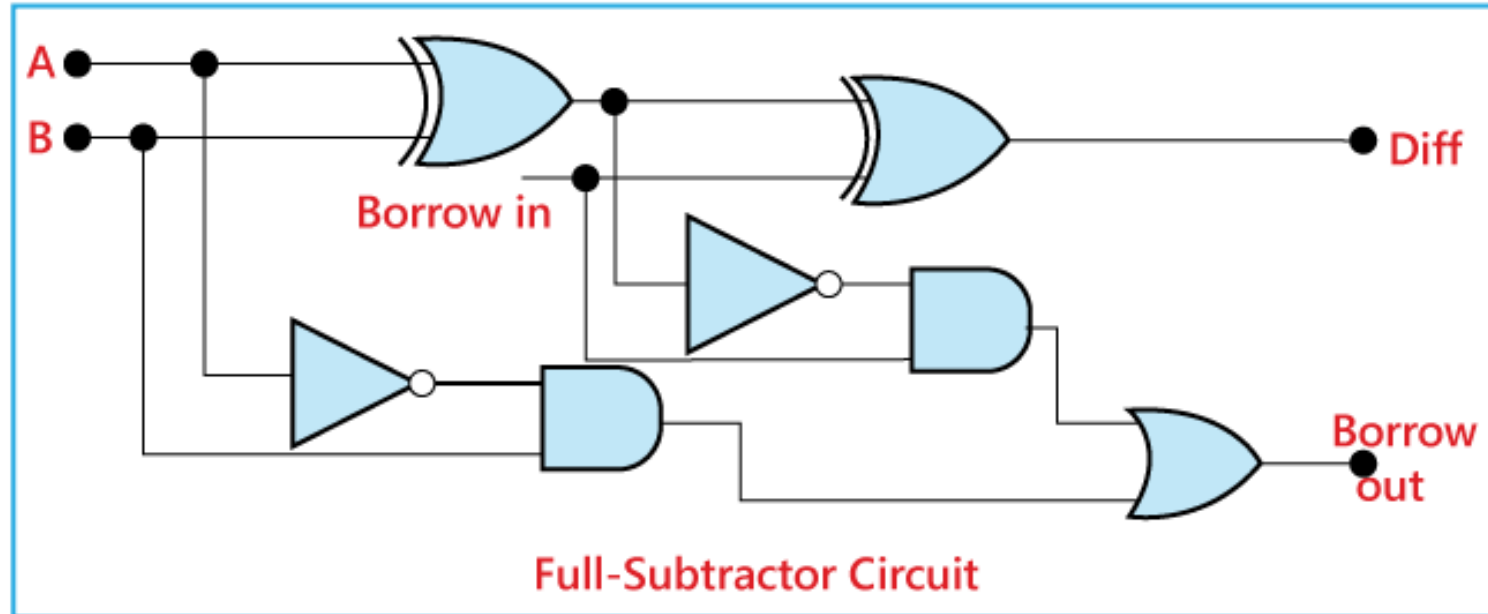
Full subtractor



$A \backslash BC$	$B'C' (00)$	$B'C (01)$	$BC (11)$	$BC' (10)$
$A' (0)$	0	1	1	1
$A (1)$	0	0	1	0

$$\text{Borrow} = A'C + BC + A'B$$

Full subtractor

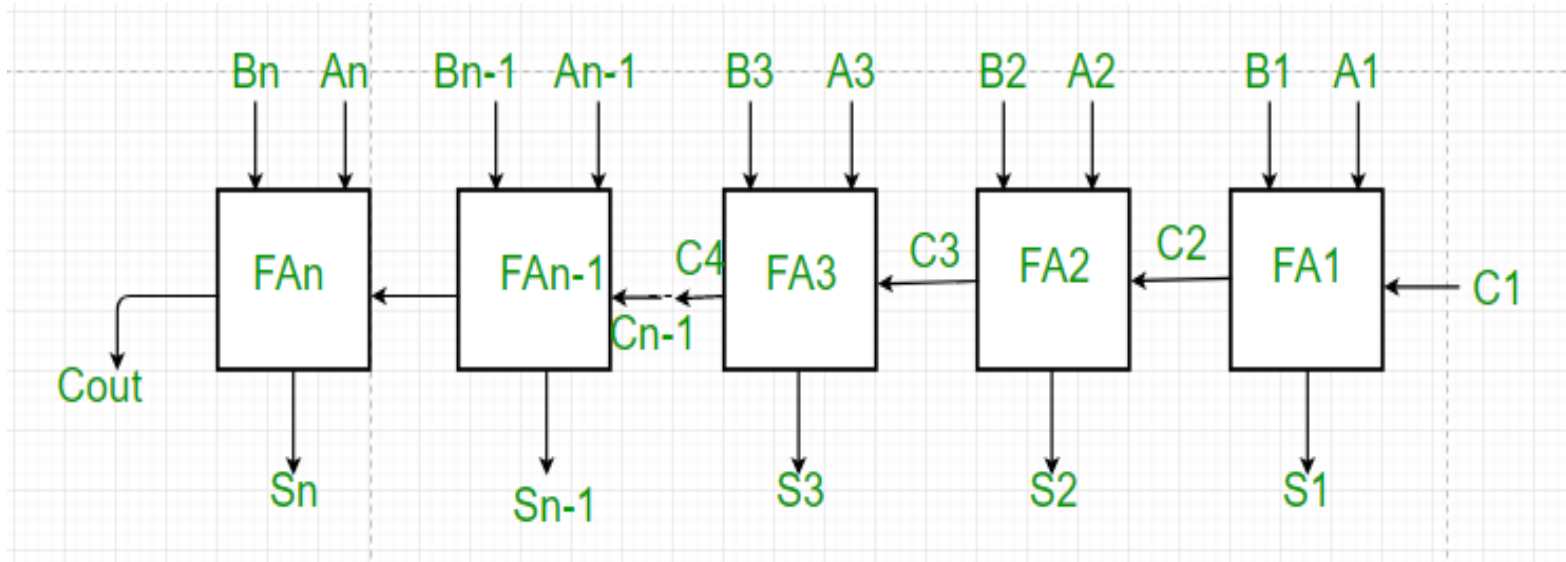


Parallel Adder (4-bit)

- Example:

$$\begin{array}{r} C_4 C_3 C_2 C_1 \leftarrow \text{Initial Carry} \\ 0 \ 0 \ 1 \ 0 \\ 1 \ 0 \ 0 \ 1 \leftarrow \text{Input A (A}_4\text{A}_3\text{A}_2\text{A}_1\text{)} \\ + 1 \ 1 \ 0 \ 1 \leftarrow \text{Input B (B}_4\text{B}_3\text{B}_2\text{B}_1\text{)} \\ \hline C_{\text{out}} \rightarrow 1 \ 0 \ 1 \ 1 \ 0 \leftarrow \text{Sum} \\ S_4 S_3 S_2 S_1 \end{array}$$

Parallel Adder (n-bit)

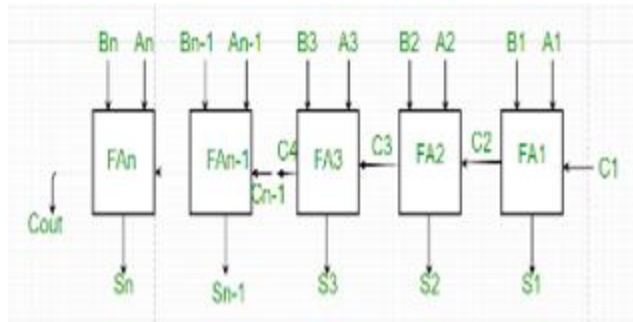


It is also known as ripple carry adder

Parallel Adder (n-bit) - Working

1. As shown in the figure, firstly the full adder FA1 adds A_1 and B_1 along with the carry C_1 to generate the sum S_1 (the first bit of the output sum) and the carry C_2 which is connected to the next adder in chain.
2. Next, the full adder FA2 uses this carry bit C_2 to add with the input bits A_2 and B_2 to generate the sum S_2 (the second bit of the output sum) and the carry C_3 which is again further connected to the next adder in chain and so on.
3. The process continues till the last full adder FA_n uses the carry bit C_n to add with its input A_n and B_n to generate the last bit of the output along last carry bit C_{out} .

Parallel Adder



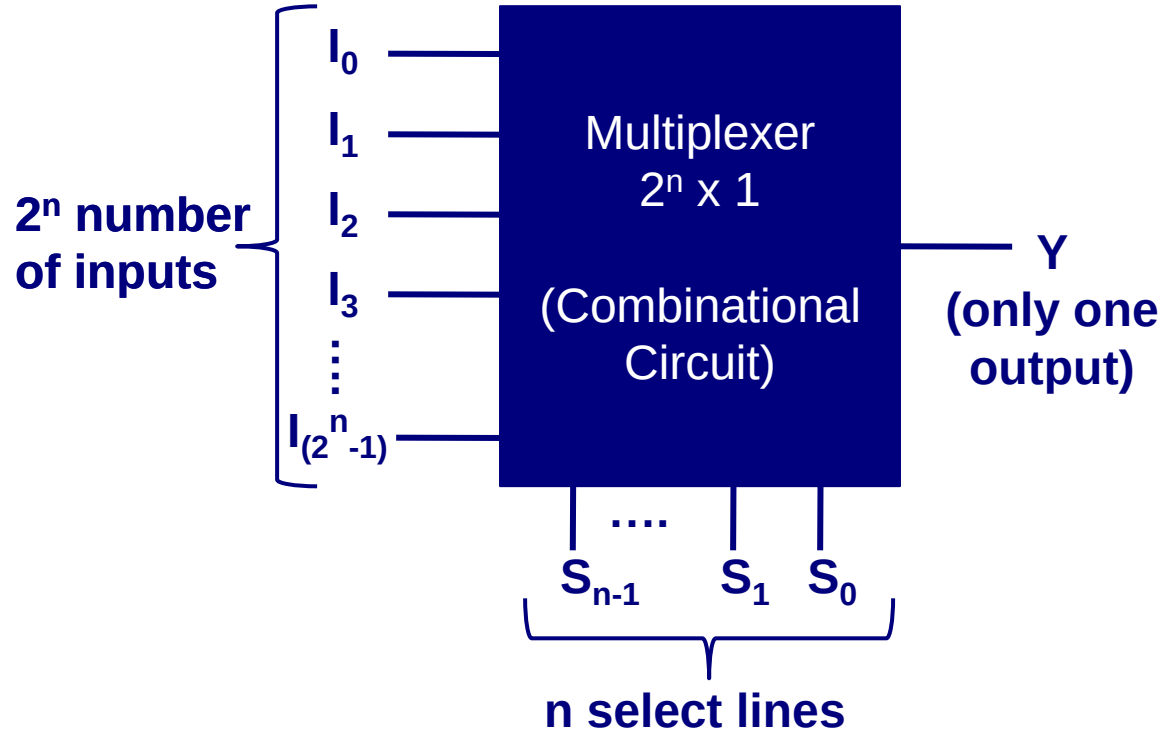
Advantages

- The parallel adder/subtractor performs the addition operation faster as compared to serial adder/subtractor.
- Time required for addition does not depend on the number of bits.
- The output is in parallel form i.e all the bits are added/subtracted at the same time.
- It is less costly.

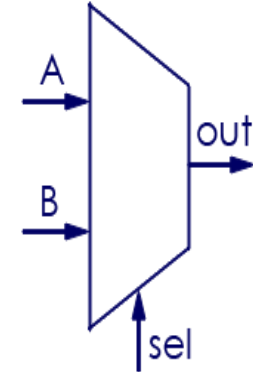
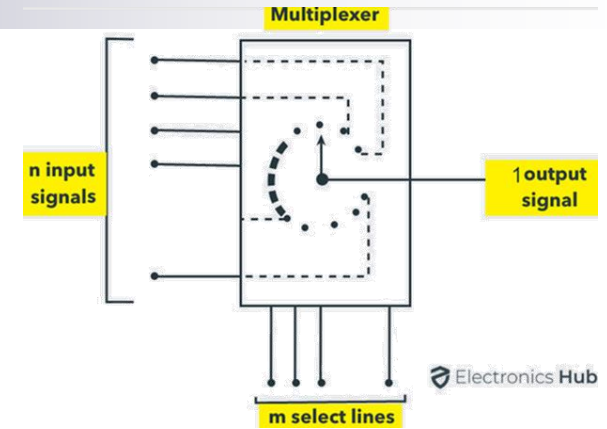
Disadvantages

- Each adder has to wait for the carry which is to be generated from the previous adder in chain.
- The propagation delay(delay associated with the travelling of carry bit) is found to increase with the increase in the number of bits to be added.

Multiplexer (MUX)

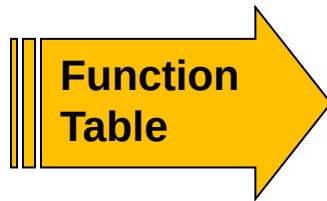
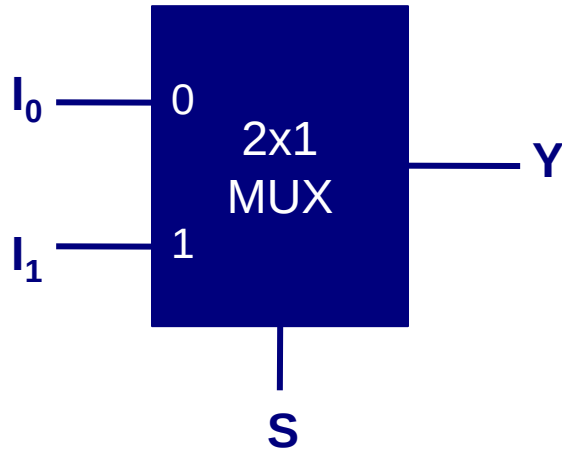


It is also called Data Selector

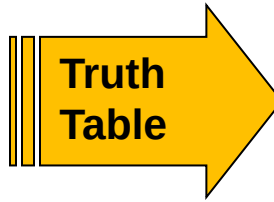


Symbolic
Representation 26

2x1 Multiplexer



S	Y
0	I_0
1	I_1



S	I_0	I_1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$$Y = S'I_0 + SI_1$$

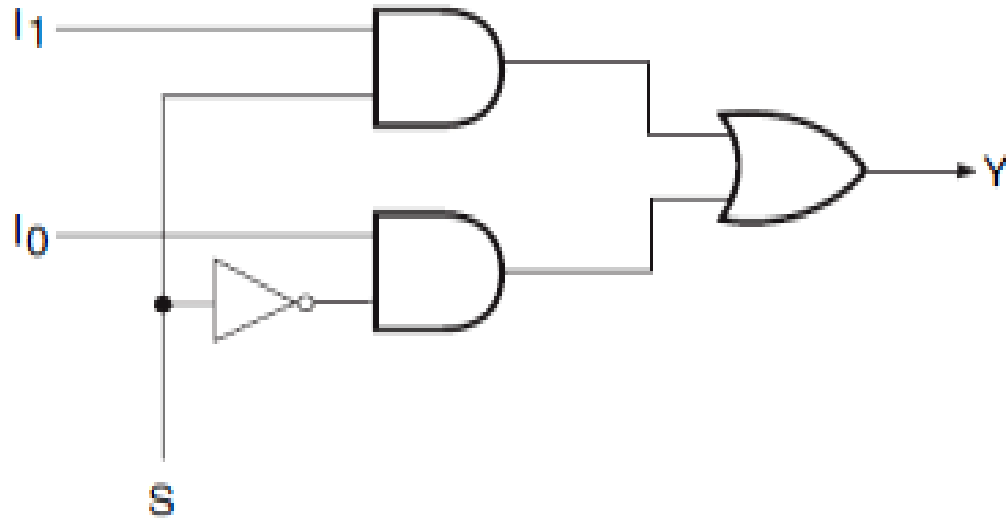
2x1 Multiplexer

K-map
for
2x1 MUX

$S \backslash I_0 I_1$	$I_0' I_1'$ (00)	$I_0' I_1$ (01)	$I_0 I_1$ (11)	$I_0 I_1'$ (10)
S' (0)	0	0	1	1
S (1)	0	1	1	0

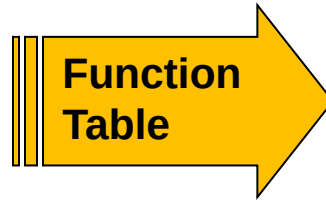
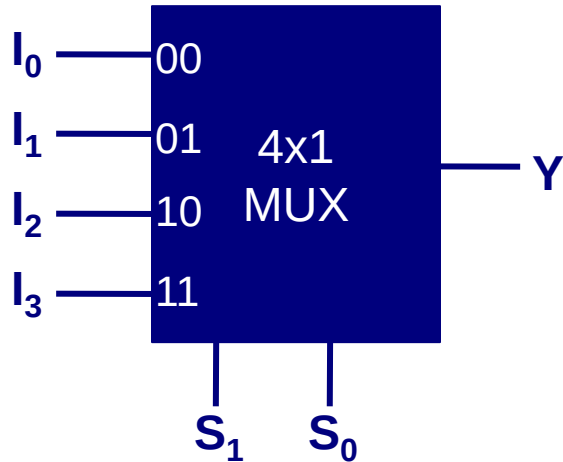
$$Y = S'I_0 + SI_1$$

2x1 Multiplexer



$$Y = S'I_0 + SI_1$$

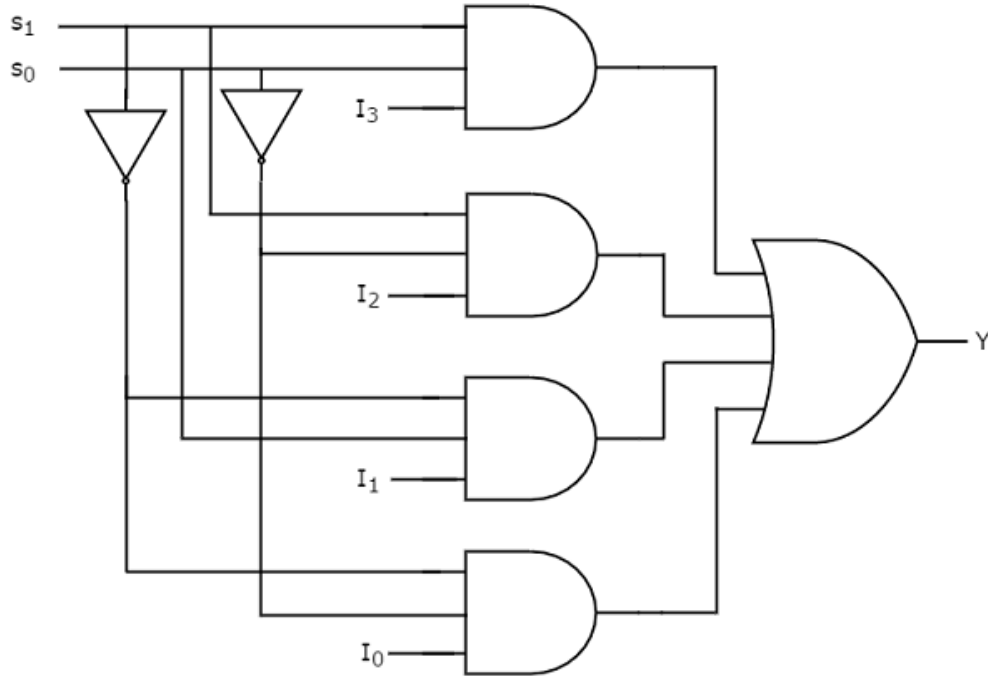
4x1 Multiplexer



S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$$

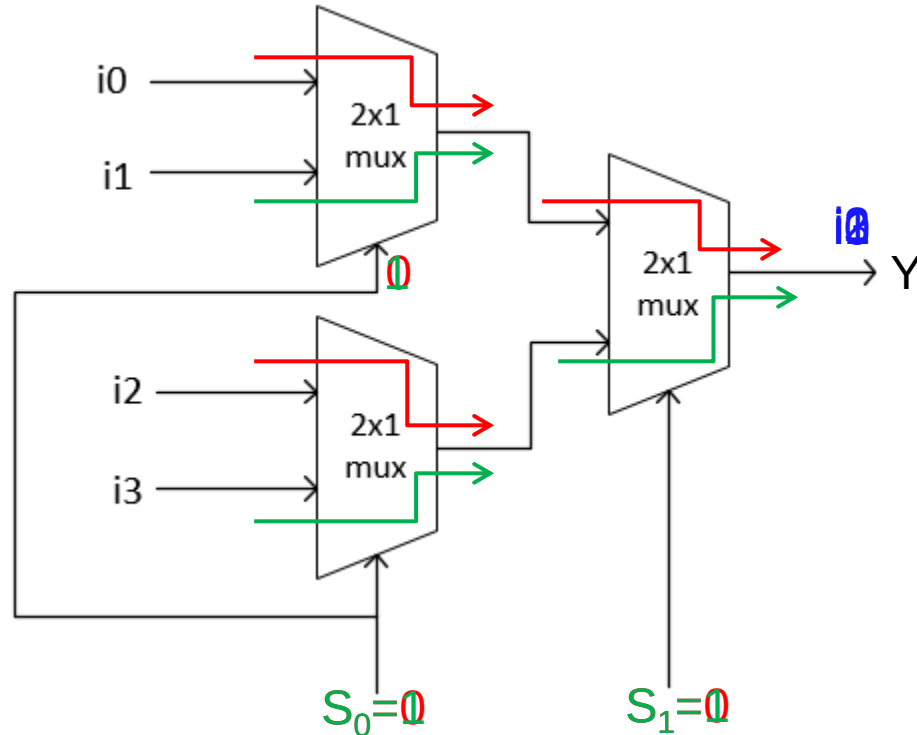
4x1 Multiplexer



$$Y = S_1' S_0' I_0 + S_1' S_0 I_1 + S_1 S_0' I_2 + S_1 S_0 I_3$$

Dr. Nilesh Patil and Mr. Shiraz Husain

4x1 MUX using 2x1 MUX



S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

To implement $2^n \times 1$ MUX using 2×1 MUX, the total number of 2×1 MUX required is $(2^n - 1)$.

∴ The number of 2×1 multiplexer required to implement 16×1 MUX will be:

$$n = 16 - 1 = 15$$

Or we can follow the below steps to calculate the same:

$$1^{\text{st}} \text{ stage} = \frac{16}{2} = 8$$

$$2^{\text{nd}} \text{ stage} = \frac{8}{2} = 4$$

$$3^{\text{rd}} \text{ stage} = \frac{4}{2} = 2$$

$$4^{\text{th}} \text{ stage} = \frac{2}{2} = 1$$

The sum will give the total number of MUX required to implement 16×1 multiplexer using 2×1 , i.e.

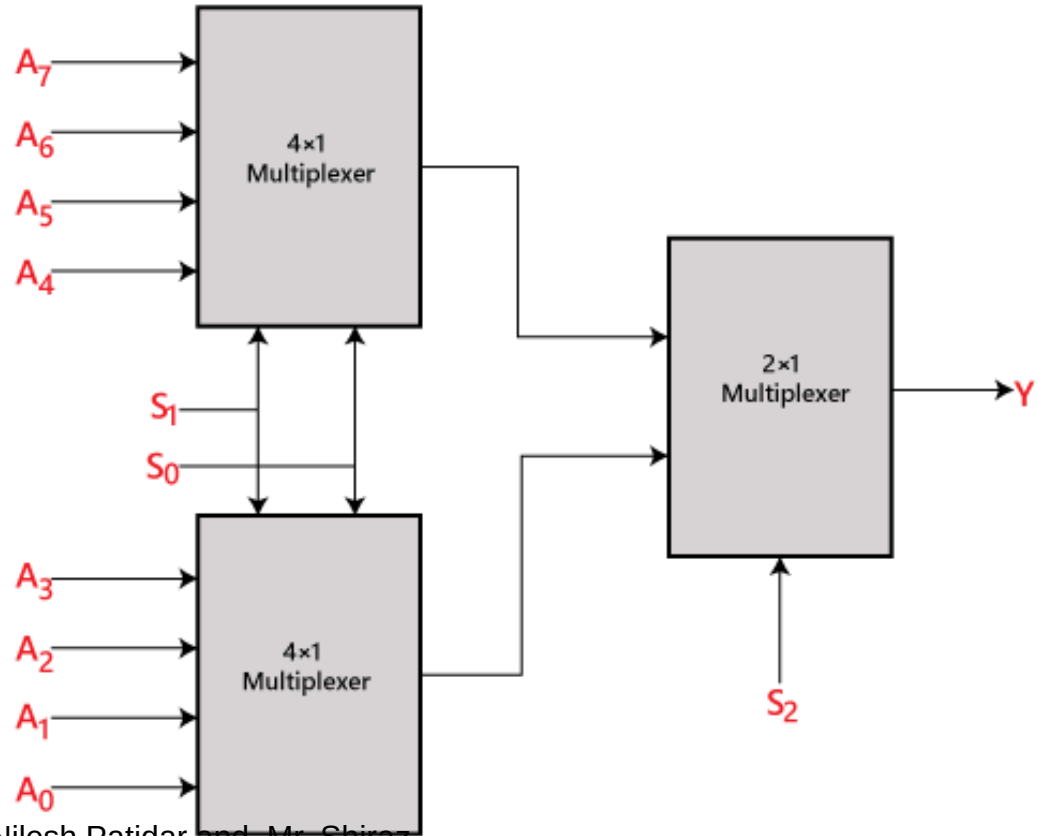
$$n = 8 + 4 + 2 + 1 = 15$$

Mux quantity as per design

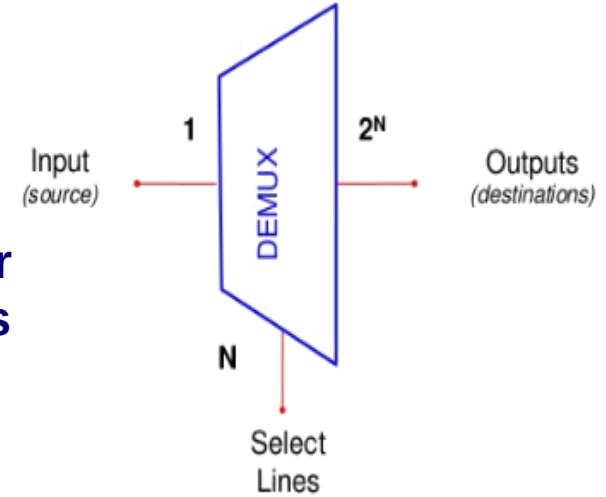
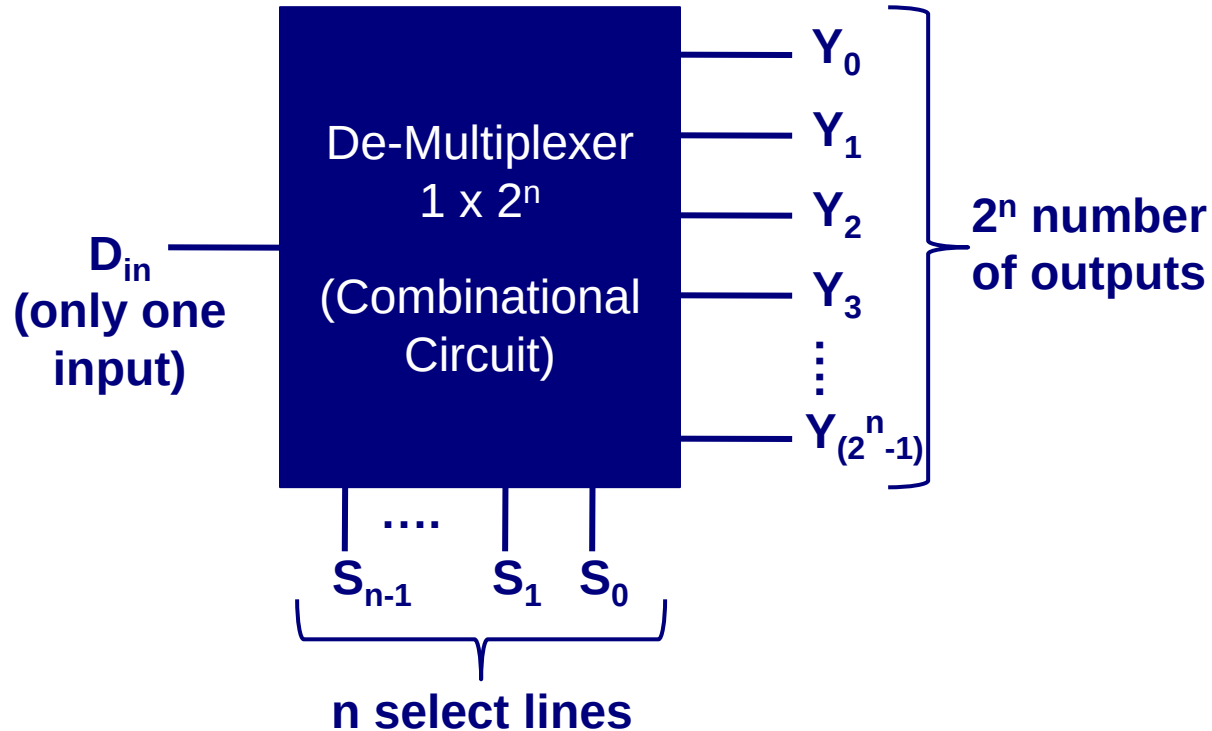
Given Mux	To be implemented	Required Quantity
2:1	16:1	$8+4+2+1 = 15$
4:1	16:1	$4+1 = 5$
4:1	64:1	$18+4+1=21$
8:1	64:1	$8+1=9$
8:1	256:1	$32+4+1=37$

8x1 Multiplexer

- To implement the 8x1 multiplexer, we need two 4x1 multiplexers and one 2x1 multiplexer. The 4x1 multiplexer has 2 selection lines, 4 inputs, and 1 output. The 2x1 multiplexer has only 1 selection line.
- For getting 8 data inputs, we need two 4x1 multiplexers. The 4x1 multiplexer produces one output. So, in order to get the final output, we need a 2x1 multiplexer.

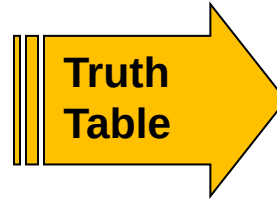
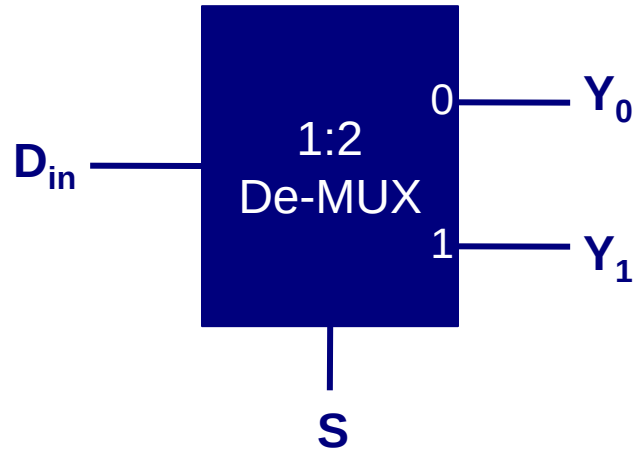


De-Multiplexer (De-MUX)

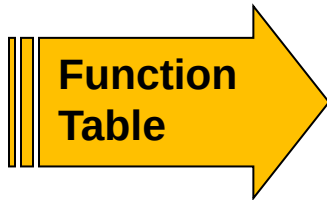


Symbolic Representation

1x2 De-Multiplexer



S	D_{in}	Y_0	Y_1
0	0	0	0
0	1	1	0
1	0	0	0
1	1	0	1

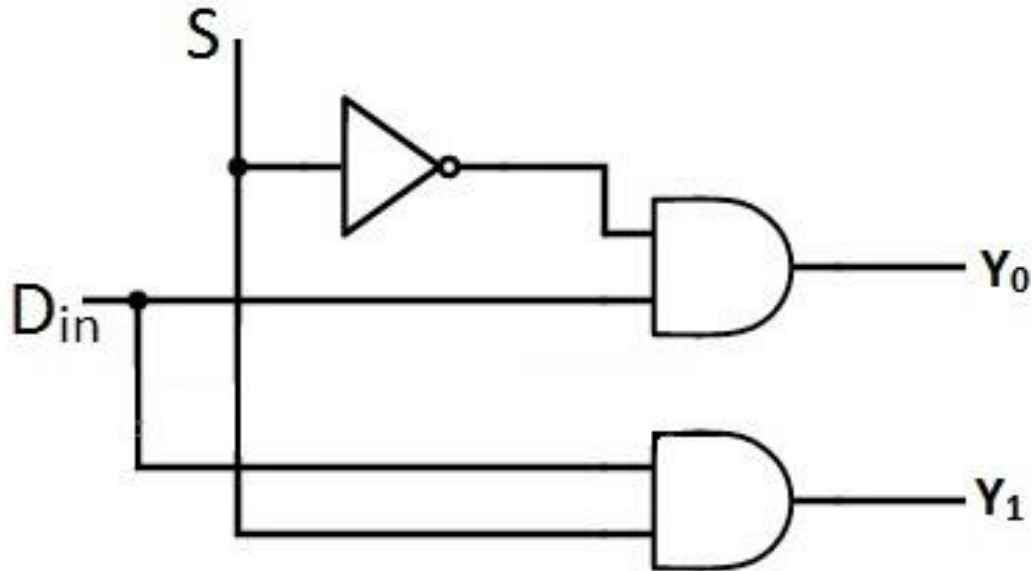


S	Y_0	Y_1
0	D_{in}	0
1	0	D_{in}

$$Y_0 = S'D_{in}$$

$$Y_1 = SD_{in}$$

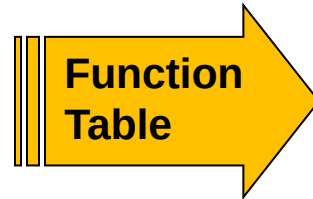
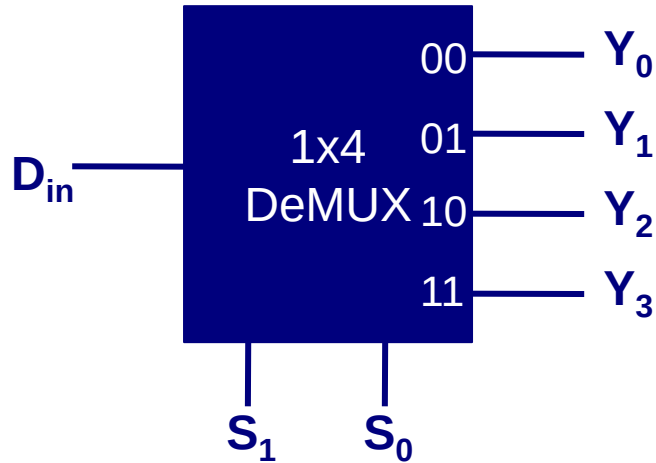
1x2 De-Multiplexer



$$Y_0 = S'D_{in}$$

$$Y_1 = SD_{in}$$

1x4 De-Multiplexer



S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	D_{in}	0	0	0
0	1	0	D_{in}	0	0
1	0	0	0	D_{in}	0
1	1	0	0	0	D_{in}

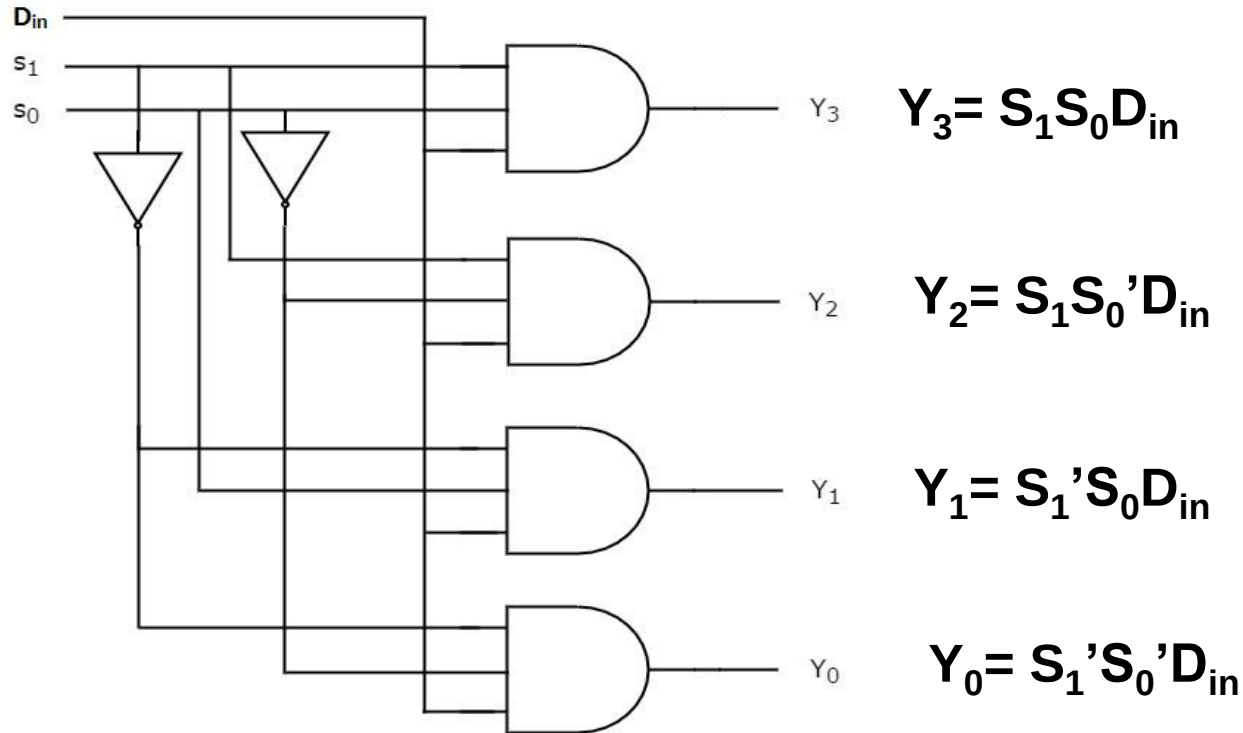
$$Y_0 = S_1' S_0' D_{in}$$

$$Y_2 = S_1 S_0' D_{in}$$

$$Y_1 = S_1' S_0 D_{in}$$

$$Y_3 = S_1 S_0 D_{in}$$

1x4 De-Multiplexer





Multiplexer (as universal logic gate)

- Multiplexer is also act as universal logic gate.
- It means that any of the logic circuit/gate can be implemented with the help of multiplexer.

Implementation using MUX (Method 1)

- Example: Design 2-input OR gate with the help of MUX.

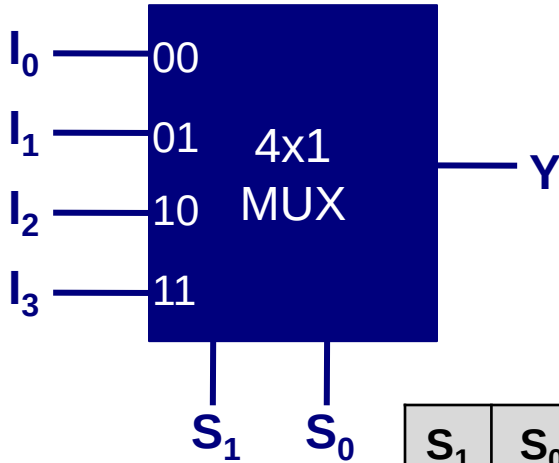
- The Boolean equation for OR gate is-

$$F(A,B) = A+B$$

- First choose the size of MUX-

- No. of select lines (n) = No. of variables in given Boolean function
 - No. of select lines (n) = 2 (A,B)
 - No. of inputs (2^n) = 4
 - It requires 4:1 MUX to implement OR gate

Implementation using MUX (Method 1)



$$F(A,B) = A+B$$

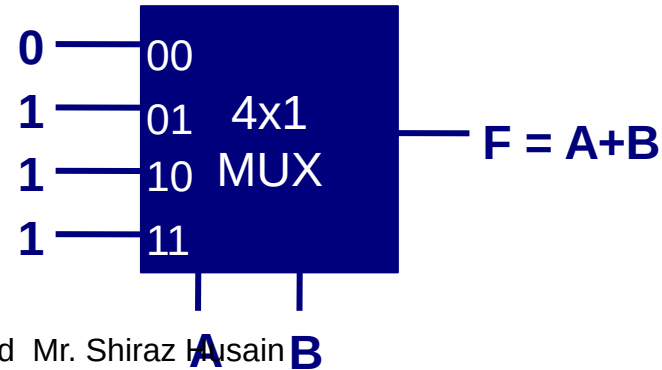
Truth Table of OR

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1

Labels I_0 , I_1 , I_2 , and I_3 are connected to the output column F for rows 0, 1, 2, and 3 respectively.

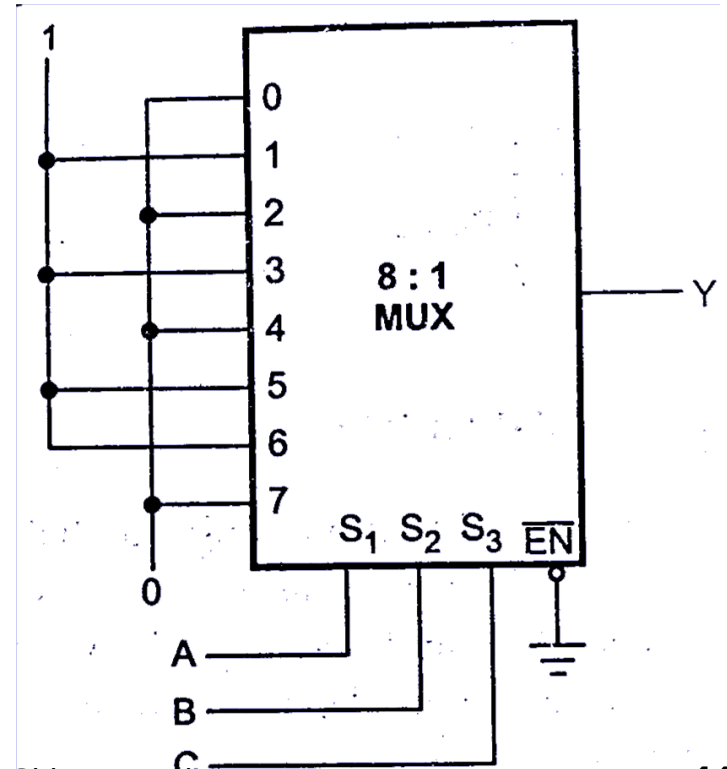
Function Table of MUX

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



Implementation using mux (Method 1)

■ $F(A,B,C) = \sum m(1,3,5,6)$



Implementation using MUX (Method 2)

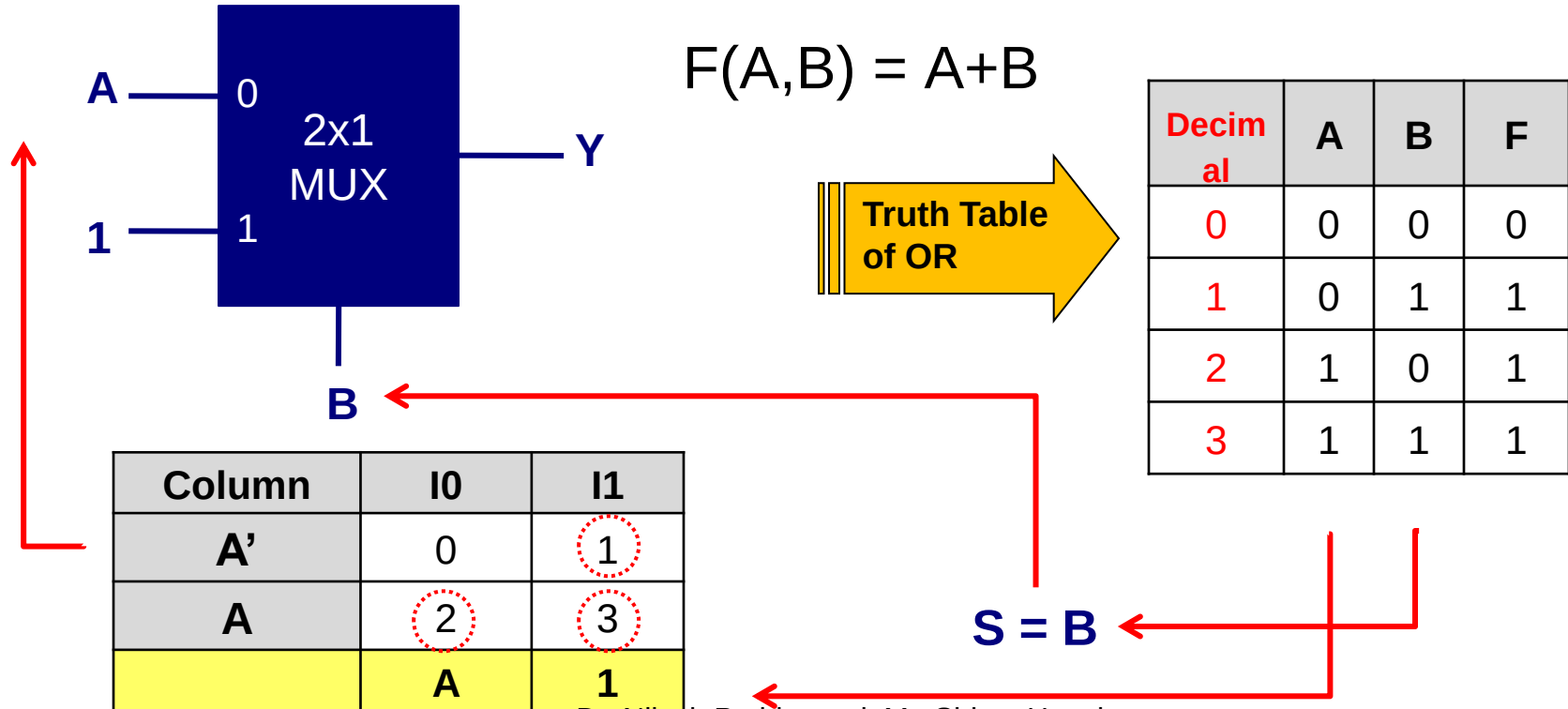
- Example: Design 2-input OR gate with the help of 2:1 MUX.

- The Boolean equation for OR gate is-

$$F(A,B) = A+B$$

- First choose the size of MUX-
 - No. of select lines (n) = No. of variables - 1
 - No. of select lines (n) = 2 - 1 = 1
 - No. of inputs (2^n) = 2
 - It requires 2:1 MUX to implement OR gate

Implementation using MUX (Method 2)



Implementation using mux (Method 2)

$$F(A,B,C) = \sum m(1,3,5,6)$$

Minterm	A	B	C	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	0
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

Fig. 3.57 (a) Truth table

	D ₀	D ₁	D ₂	D ₃	
$\bar{A}$	0	①	2	③	Row 1
A	4	⑤	⑥	7	Row 2
	0	1	A	$\bar{A}$	

Fig. 3.57 (c) Implementation table

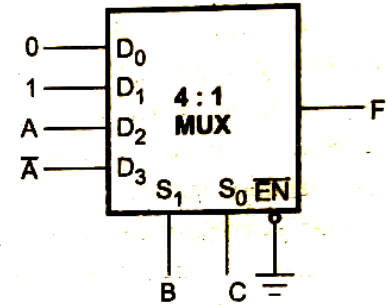


Fig. 3.57 (b) Multiplexer implementation

Implementation using MUX (Method 2)

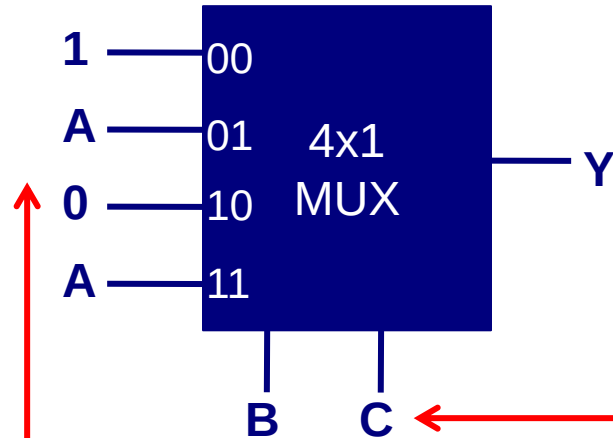
Design the following function by using MUX

$$F(A,B,C) = \Sigma m(0,4,5,7)$$

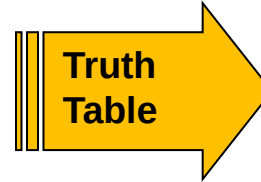
Solution: First choose the size of required MUX-

- No. of select lines (n) = No. of variables - 1
- No. of select lines (n) = 3 - 1 = 2
- No. of inputs (2^n) = 4
- It requires 4:1 MUX to implement the given function

Implementation using MUX (Method 2)



$$F(A,B,C) = \Sigma m(0,4,5,7)$$



Column	I0	I1	I2	I3
A'	0	1	2	3
A	4	5	6	7
	1	A	0	A

$$S_1 = B$$

$$S_0 = C$$

Decimal	A	B	C	F
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

Question

- Implement the following function using 8:1 Mux
 - $F(P,Q,R,S) = \sum m(0,1,3,4,8,9,15)$

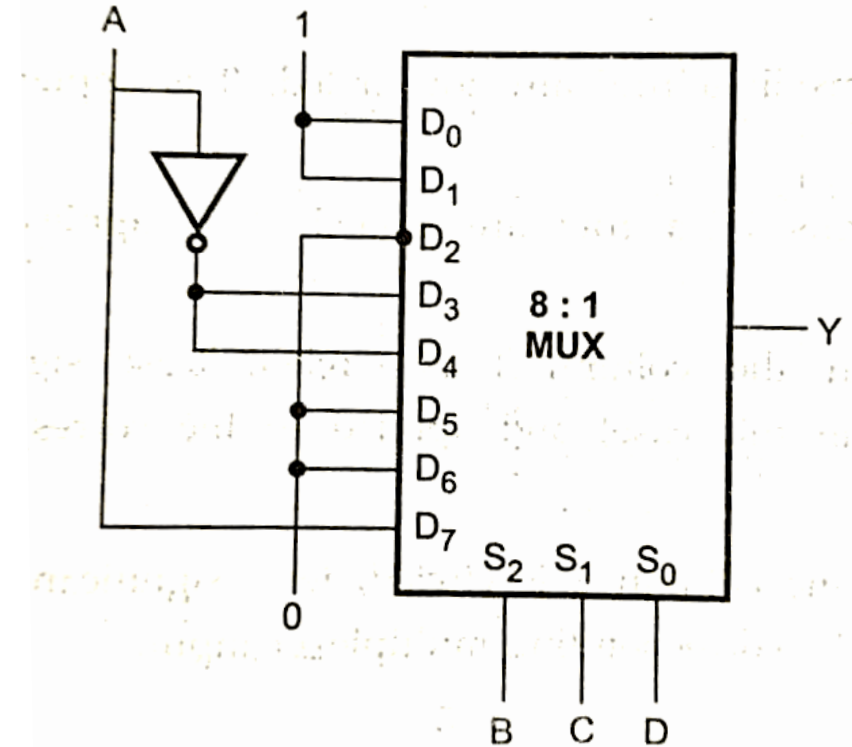
Question

- Implement the following function using 8:1 Mux

□ $F(P,Q,R,S) = \sum m(0,1,3,4,8,9,15)$

	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
$\bar{A}$	①	①	2	③	④	5	6	7
A	⑧	⑨	10	11	12	13	14	⑮
	1	1	0	$\bar{A}$	$\bar{A}$	0	0	A

(a) Implementation table



(b) Multiplexer implementation

Question

- Implement the following function using 8:1 Mux
 - $F(A,B,C,D) = \sum m(0,2,6,10,11,12,13) + d(3,8,14)$

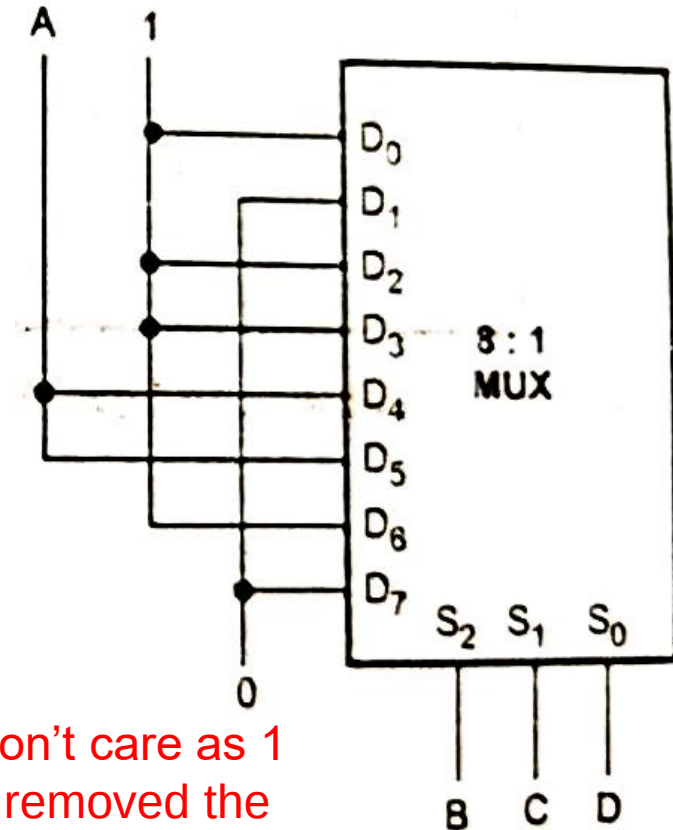
Question

- Implement the following function using 8:1 Mux

$$F(A,B,C,D) = \sum m(0,2,6,10,11,12,13) + d(3,8,14)$$

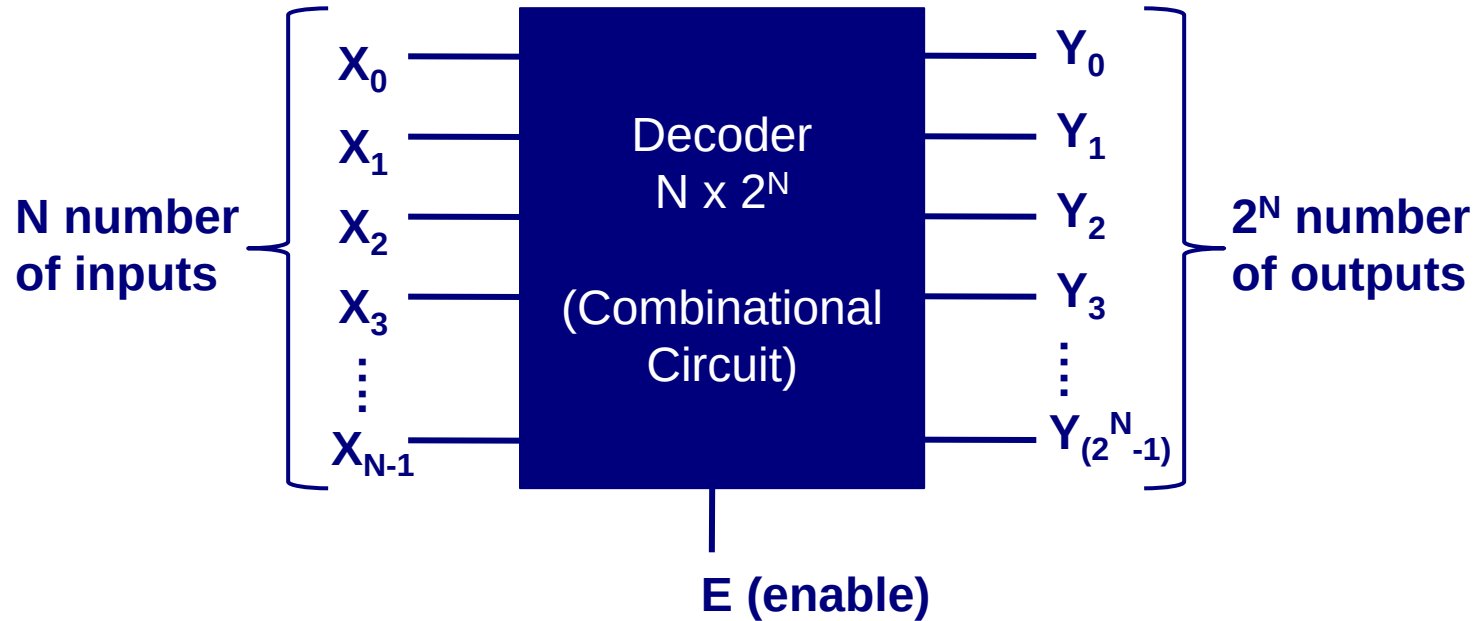
	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
$\bar{A}$	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
	1	0	1	1	A	A	1	0

Here, don't cares are treated as 1s

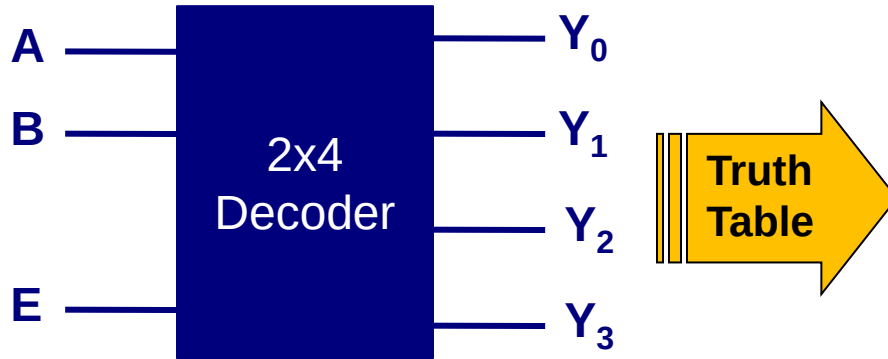


Taking Don't care as 1
we have removed the
inverter for Abar

Decoder



2x4 Decoder



$$Y_0 = EA'B'$$

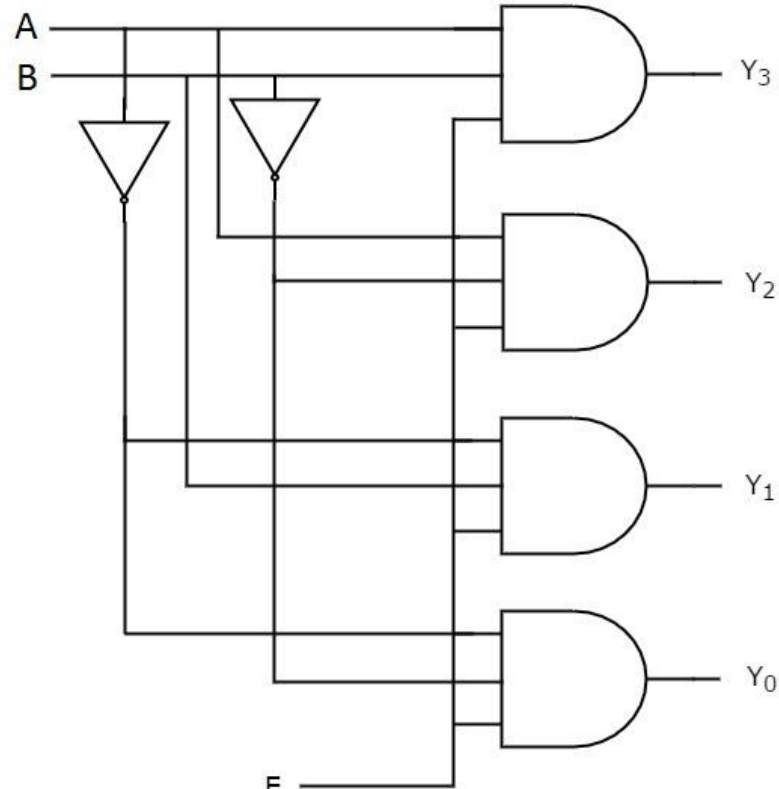
$$Y_2 = EAB'$$

$$Y_1 = EA'B$$

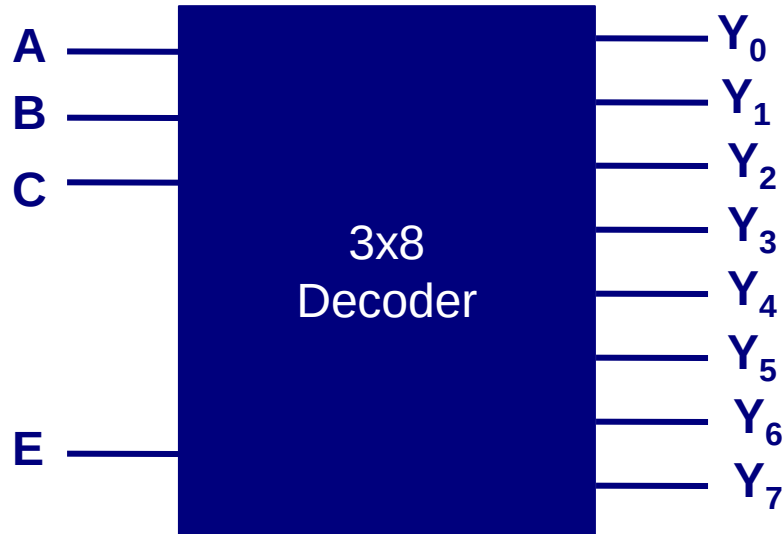
$$Y_3 = EAB$$

E	A	B	Y ₀	Y ₁	Y ₂	Y ₃
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

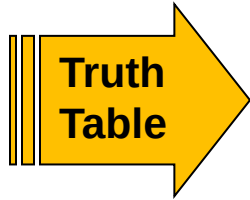
2x4 Decoder



3x8 Decoder

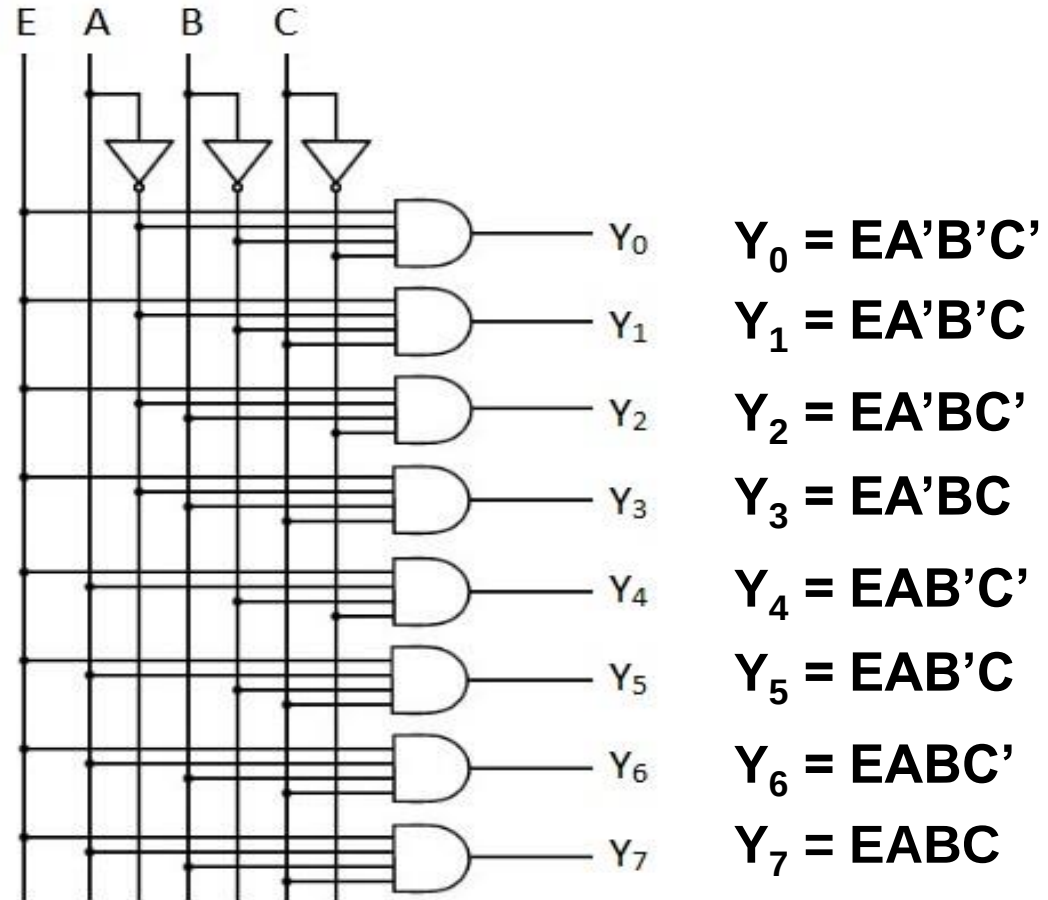


3x8 Decoder



E	A	B	C	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇
0	X	X	X	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	1	0	0	0	0	0	0
1	0	1	0	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	1	0	0	0	0
1	1	0	0	0	0	0	0	1	0	0	0
1	1	0	1	0	0	0	0	0	1	0	0
1	1	1	0	0	0	0	0	0	0	1	0
1	1	1	1	0	0	0	0	0	0	0	1

3x8 Decoder



4 to 16 Decoder

In this section, let us implement **4 to 16 decoder using 3 to 8 decoders**. We know that 3 to 8 Decoder has three inputs A_2, A_1 & A_0 and eight outputs, Y_7 to Y_0 . Whereas, 4 to 16 Decoder has four inputs A_3, A_2, A_1 & A_0 and sixteen outputs, Y_{15} to Y_0

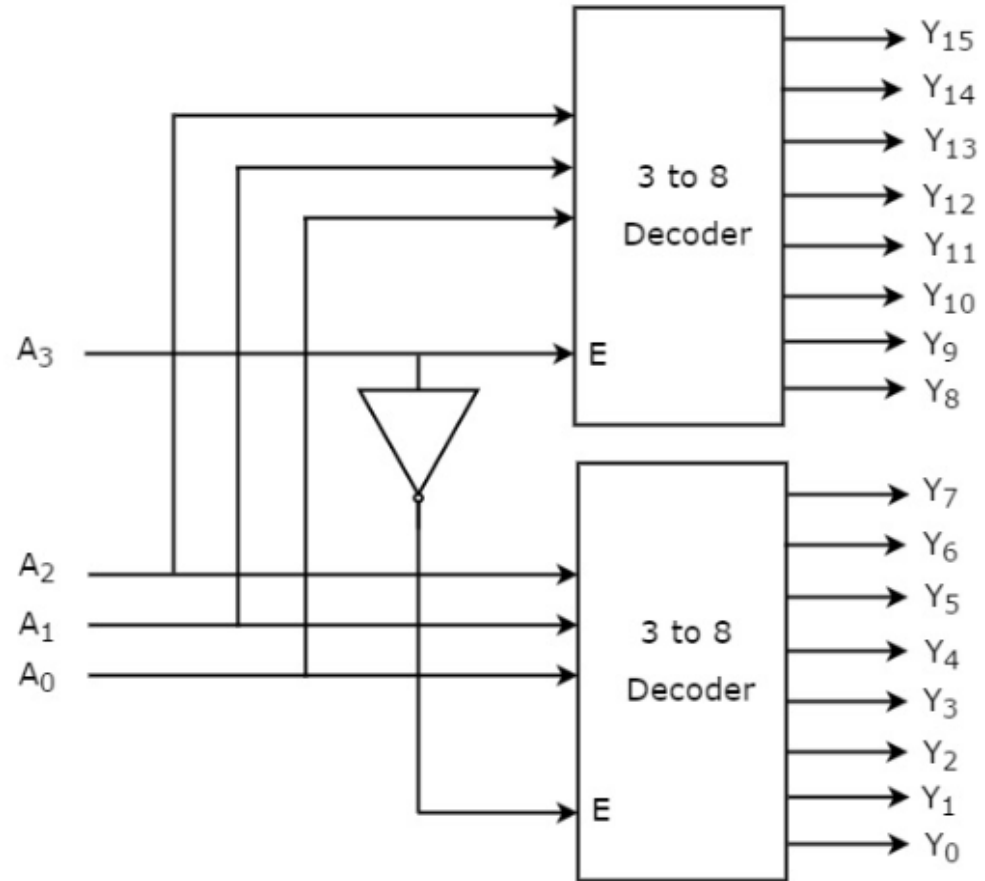
We know the following formula for finding the number of lower order decoders required.

$$\text{Required number of lower order decoders} = \frac{m_2}{m_1}$$

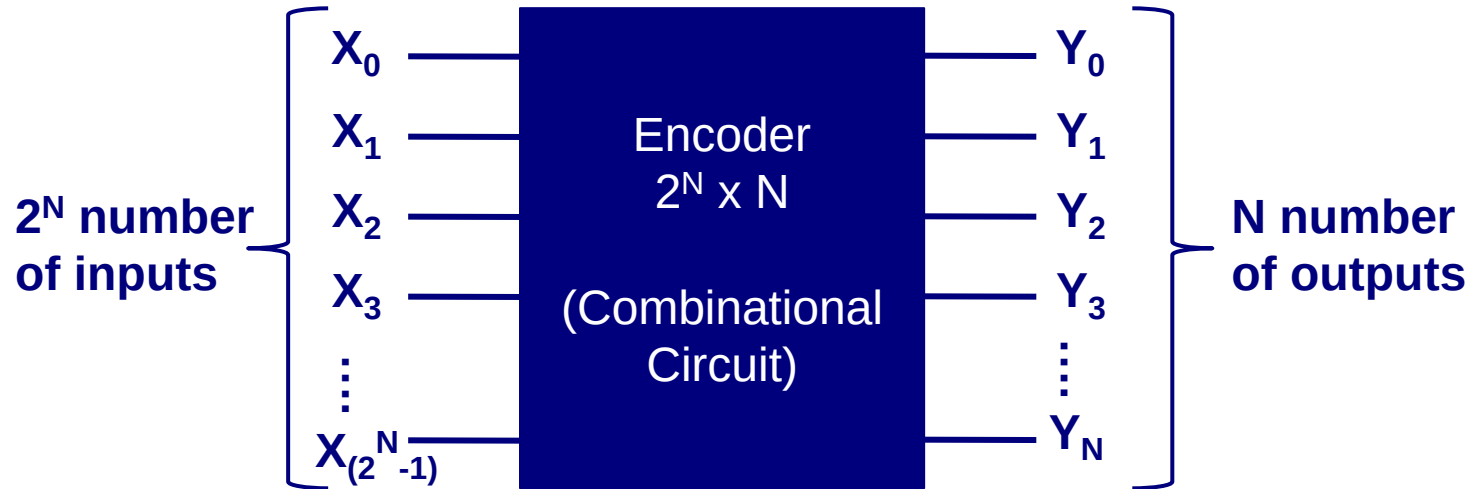
Substitute, $m_1 = 8$ and $m_2 = 16$ in the above formula.

$$\text{Required number of 3 to 8 decoders} = \frac{16}{8} = 2$$

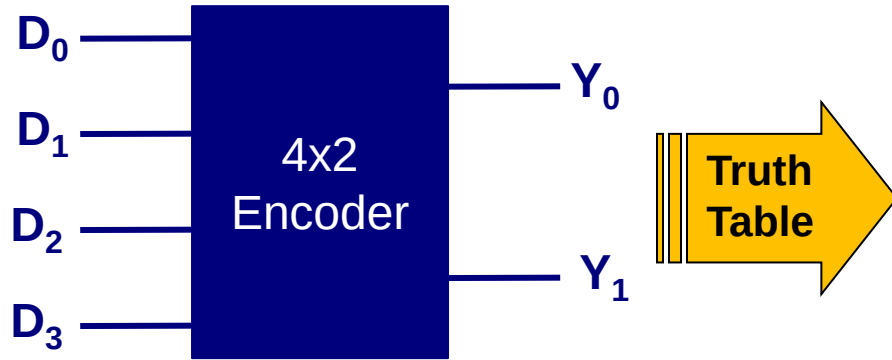
4:16 Decoder using 3:8 Decoder



Encoder



4x2 Encoder



$$Y_0 = D_1 + D_3$$

$$Y_1 = D_2 + D_3$$

D_0	D_1	D_2	D_3	Y_1	Y_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1
X	X	X	X	X	X

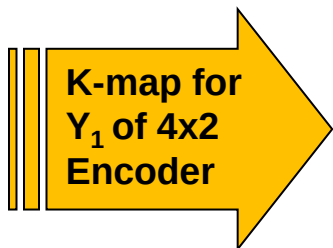
4x2 Encoder

K-map for Y_0 of 4x2 Encoder

D_2D_3 D_0D_1				
	0	1	3	2
	4	5	7	6
	12	13	15	14
	8	9	10	11

$$Y_0 = D_1 + D_3$$

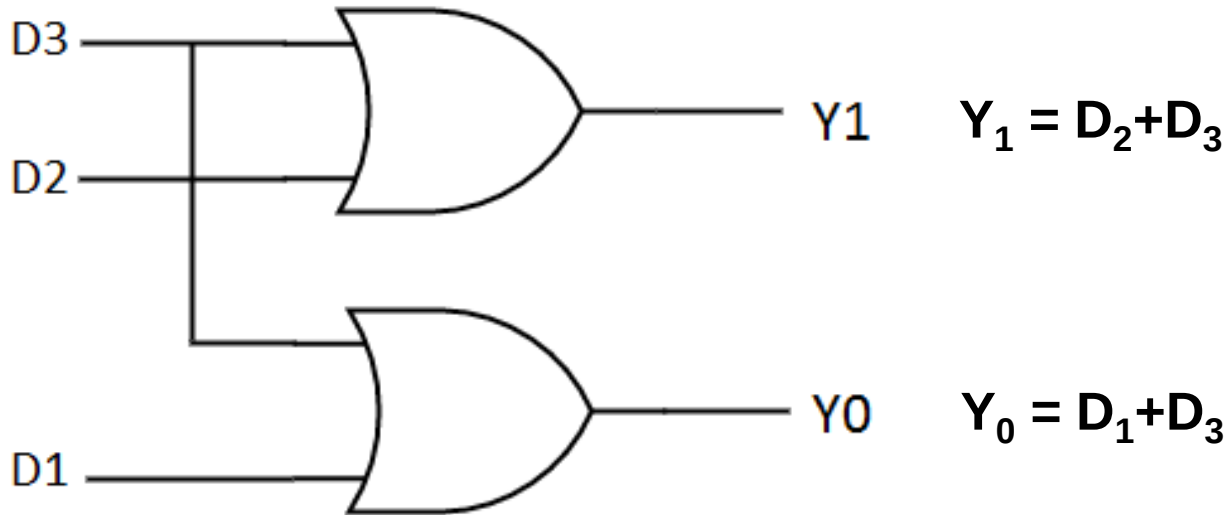
4x2 Encoder



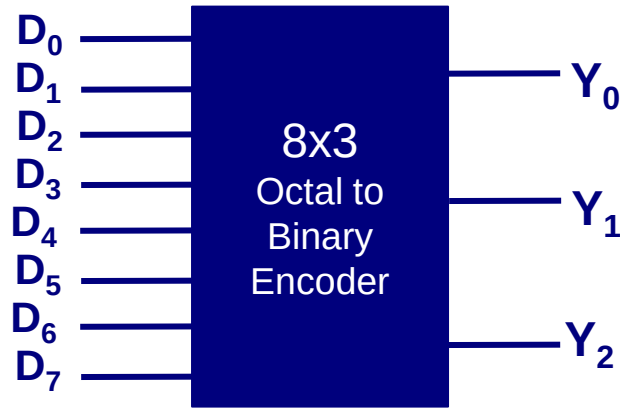
D_2D_3 D_0D_1				
	0	1	3	2
	4	5	7	6
	12	13	15	14
	8	9	10	11

$$Y_1 = D_2 + D_3$$

4x2 Encoder



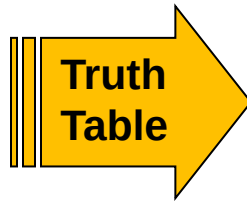
8x3 Encoder (Octal to Binary)



$$Y_0 = D_1 + D_3 + D_5 + D_7$$

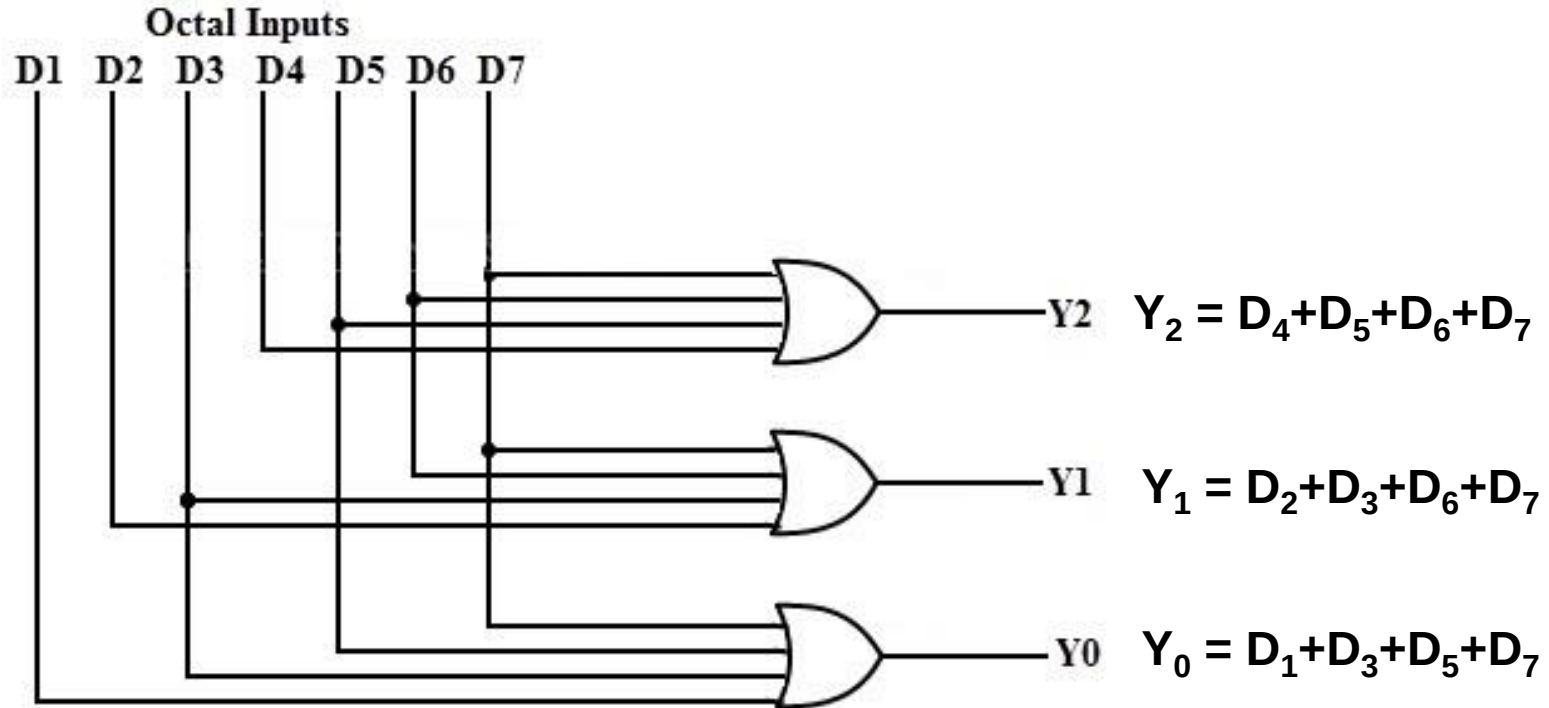
$$Y_1 = D_2 + D_3 + D_6 + D_7$$

$$Y_2 = D_4 + D_5 + D_6 + D_7$$



D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	Y_2	Y_1	Y_0
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1
X	X	X	X	X	X	X	X	X	X	X

8x3 Encoder (Octal to Binary)



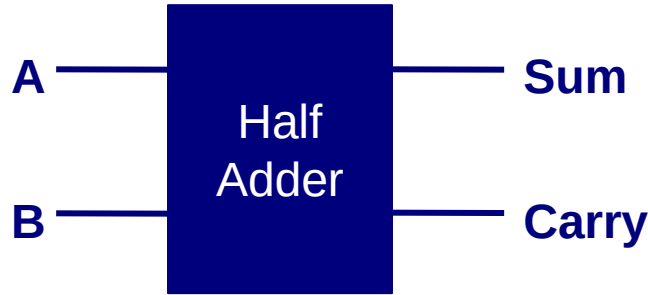


Decoder (as universal logic gate)

- Decoder is also act as universal logic gate.
- It means that any of the logic circuit/gate can be implemented with the help of Decoder.

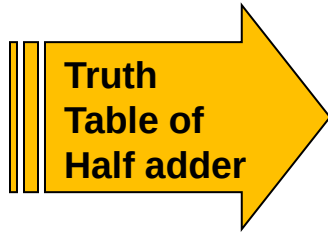
Implementation of Half adder using decoder

- First, find out the no. of inputs required.



- So, half adder has 2 inputs.
- It means 2x4 decoder is required to design a half adder.

Implementation of Half adder using decoder

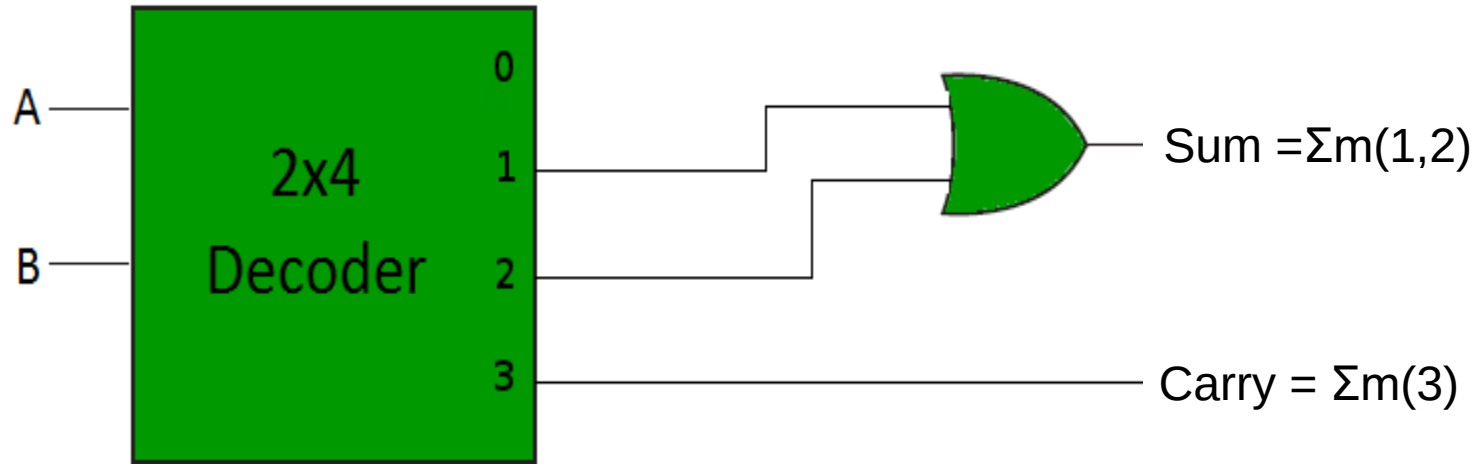


Decimal	A	B	Sum	Carry
0	0	0	0	0
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1

$$\text{Sum} = \Sigma m(1,2)$$

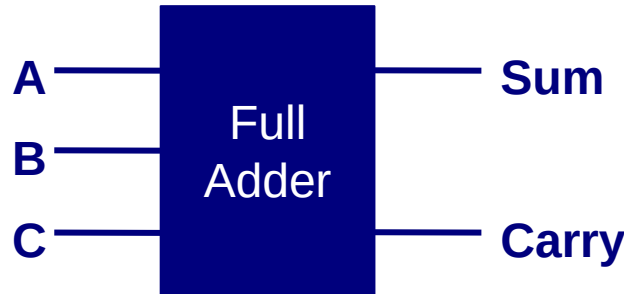
$$\text{Carry} = \Sigma m(3)$$

Implementation of Half adder using decoder



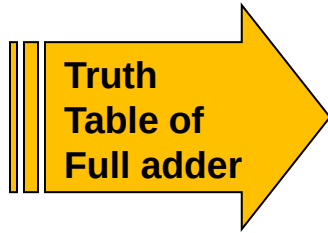
Decoder (as universal logic gate)

- Implement a full adder with the help of decoder.
 - First, find out the no. of inputs required.



- So, full adder has 3 inputs.
- It means 3x8 decoder is required to design a full adder.

Implementation of full adder using decoder

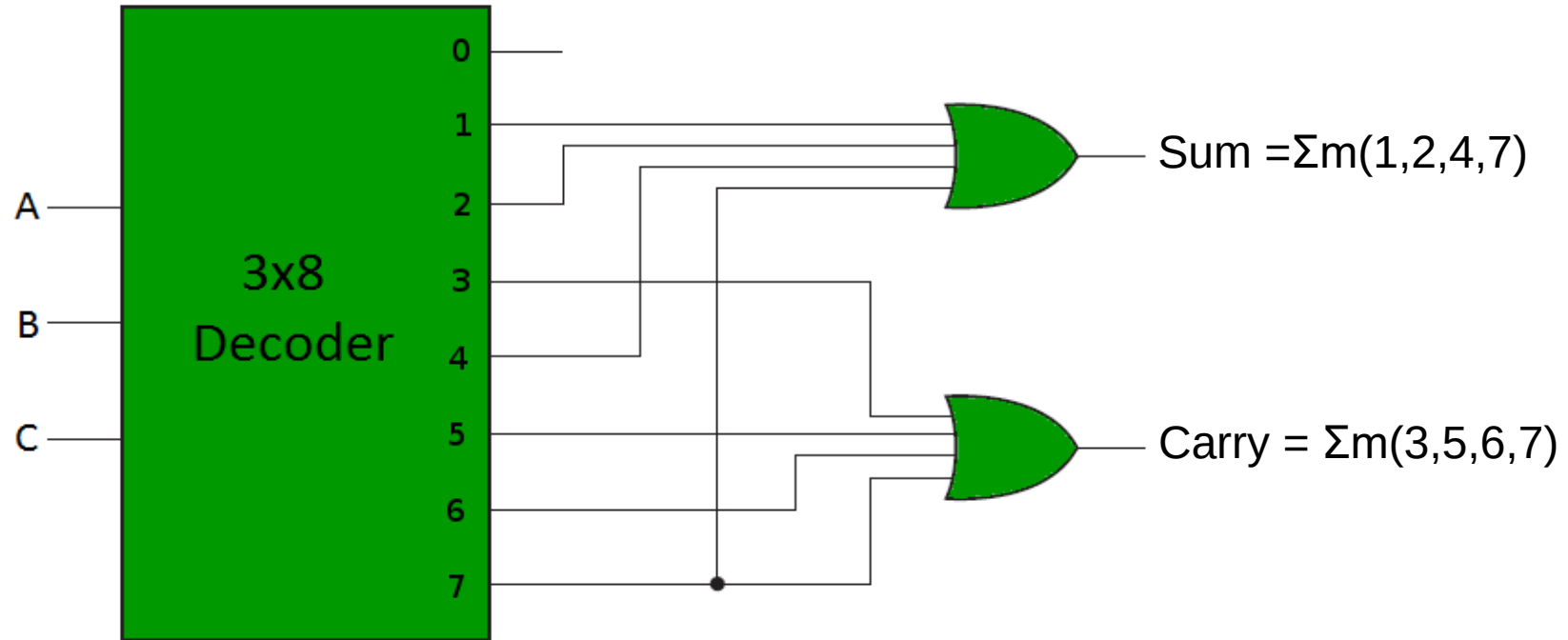


$$\text{Sum} = \Sigma m(1,2,4,7)$$

$$\text{Carry} = \Sigma m(3,5,6,7)$$

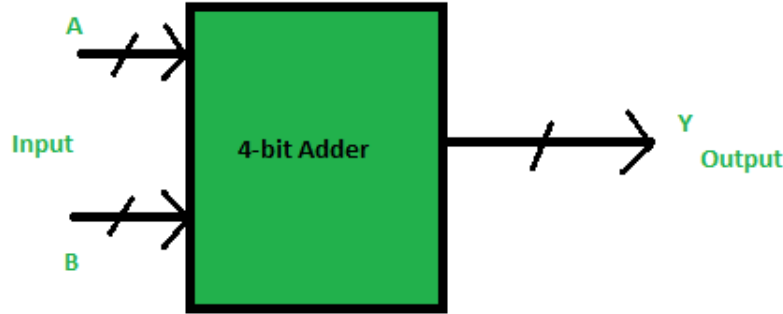
Decimal	A	B	C	Sum	Carry
0	0	0	0	0	0
1	0	0	1	1	0
2	0	1	0	1	0
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1

Implementation of full adder using decoder



BCD Adder

- BCD stand for binary coded decimal. (Range from 0 to 9)



Maximum value of any BCD numbers is 9. So, maximum sum of two BCD numbers are 18 (without any carry) and 19 (in case of previous carry)

$$\text{BCD Sum} = 9 + 9 + 1(\text{carry}) = 19$$

BCD Adder

- Example: Add two BCD numbers.

$$\begin{array}{rcl} & 1\ 0\ 0\ 1 & \leftarrow 9 \text{ (in BCD)} \\ + & 0\ 1\ 0\ 1 & \leftarrow 5 \text{ (in BCD)} \\ \hline \text{Binary Sum} & 1\ 1\ 1\ 0 & \leftarrow 14 \text{ (in Decimal)} \\ & +\ 0\ 1\ 1\ 0 & \leftarrow \text{add 6 (in case of sum} > 9) \\ \hline \text{BCD Sum} & 0\ 0\ 0\ 1\ 0\ 1\ 0\ 0 & \leftarrow 1\ 4 \text{ (in BCD)} \end{array}$$

BCD Adder

Decimal	Binary Sum					BCD Sum				
	C'	S3'	S2'	S1'	S0'	C	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

BCD Adder

We are adding “0110” (=6) only to the second half of the table.

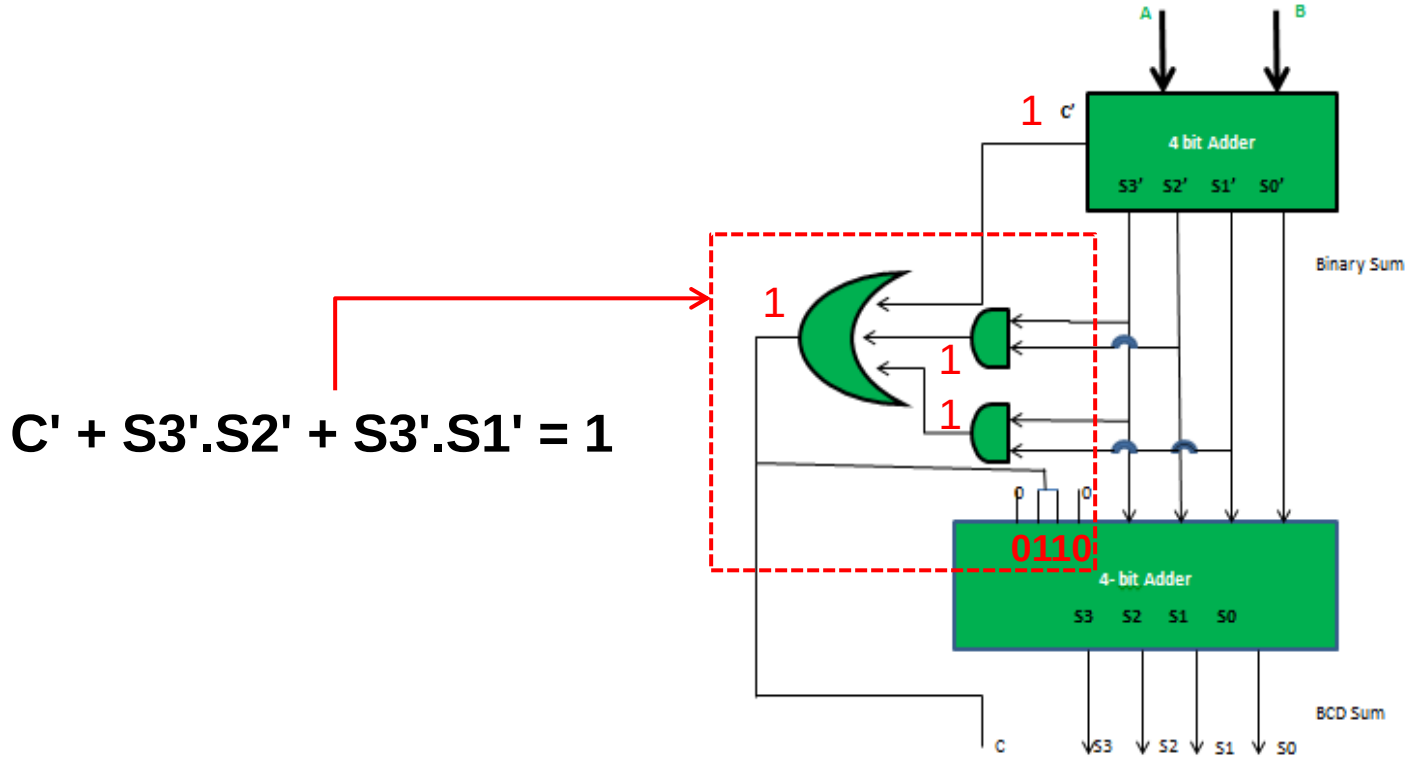
The conditions are:

- If $C' = 1$ (Satisfies 16-19)
- If $S3'.S2' = 1$ (Satisfies 12-15)
- If $S3'.S1' = 1$ (Satisfies 10 and 11)

■ So, our logic is

$$C' + S3'.S2' + S3'.S1' = 1 \text{ (used to generate 6)}$$

BCD Adder (Implementation)





Thank You